

From Sensors to Actuators

Vladimir Toporkov

May 17, 2026



System Overview



System Overview



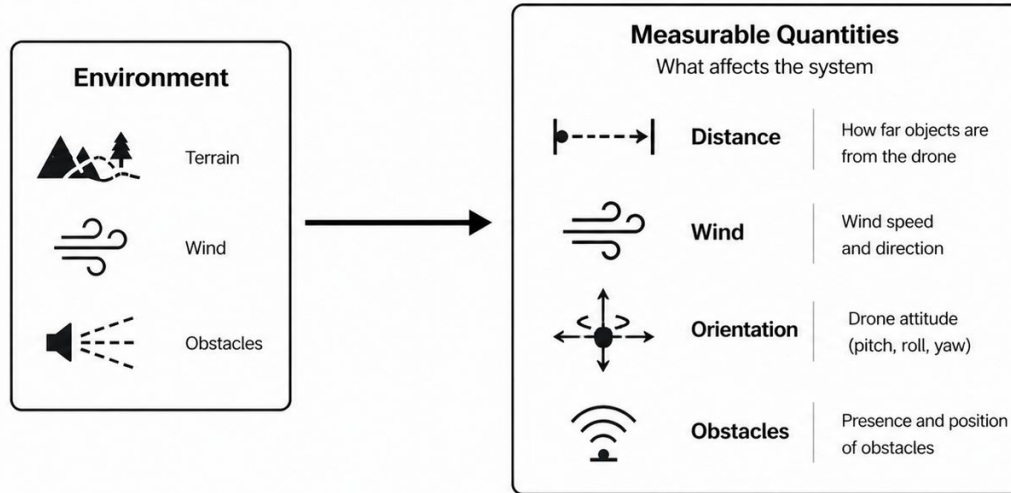
System Overview



System Overview



Environment

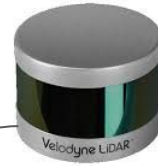


From Quantities to Sensors

Measured quantity	Sensor
--------------------------	---------------

From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic



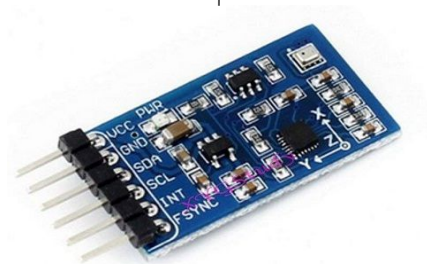
From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer



From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU



From Quantities to Sensors

Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

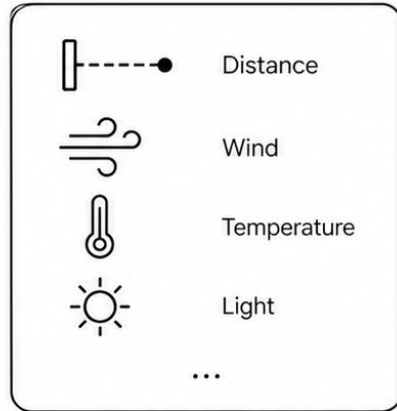


From Quantities to Sensors

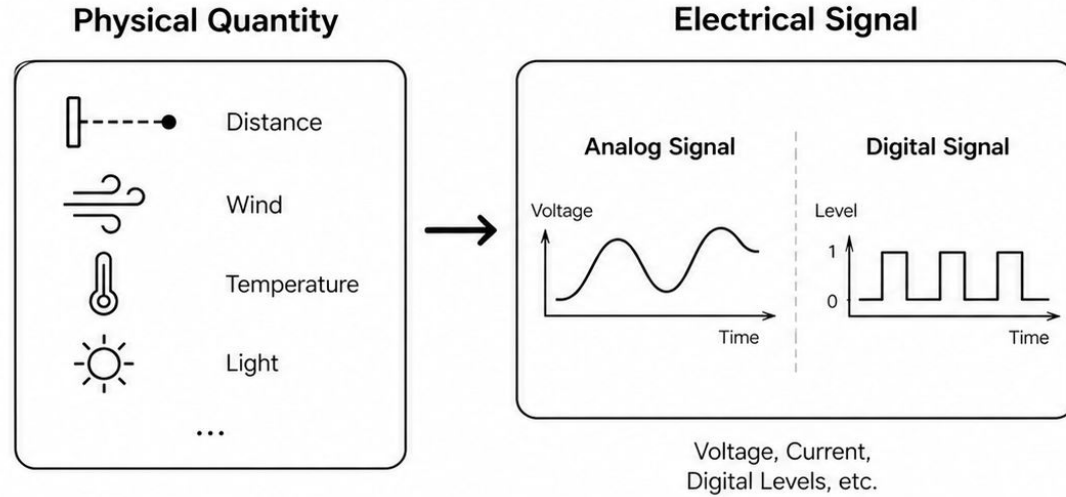
Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

What is a sensor?

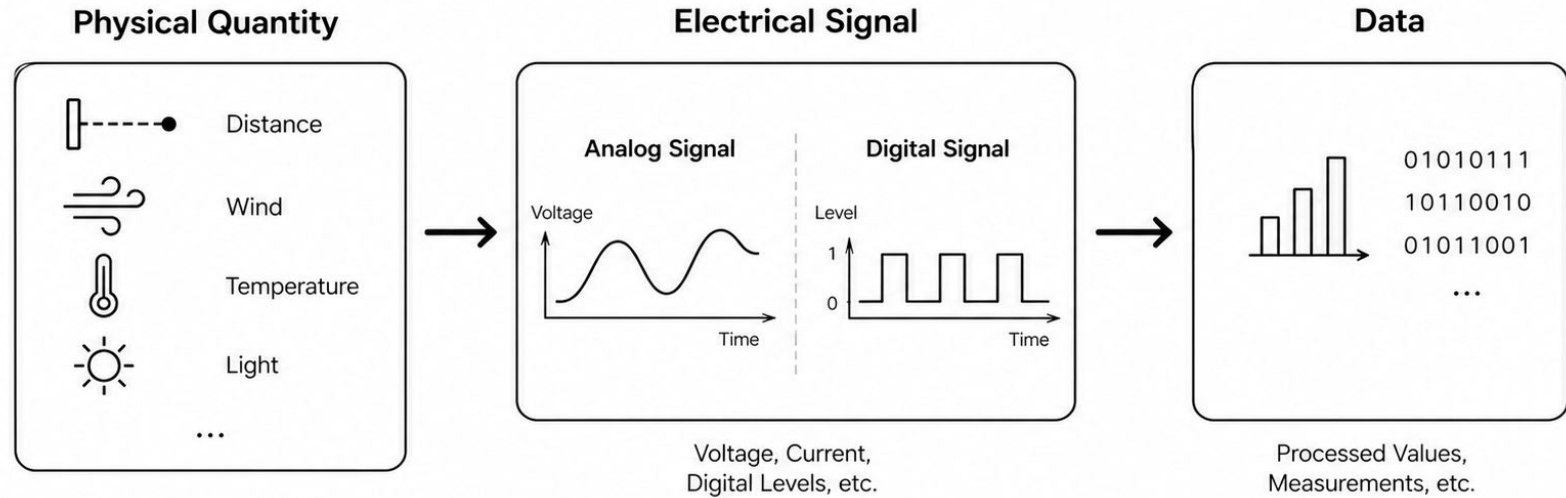
Physical Quantity



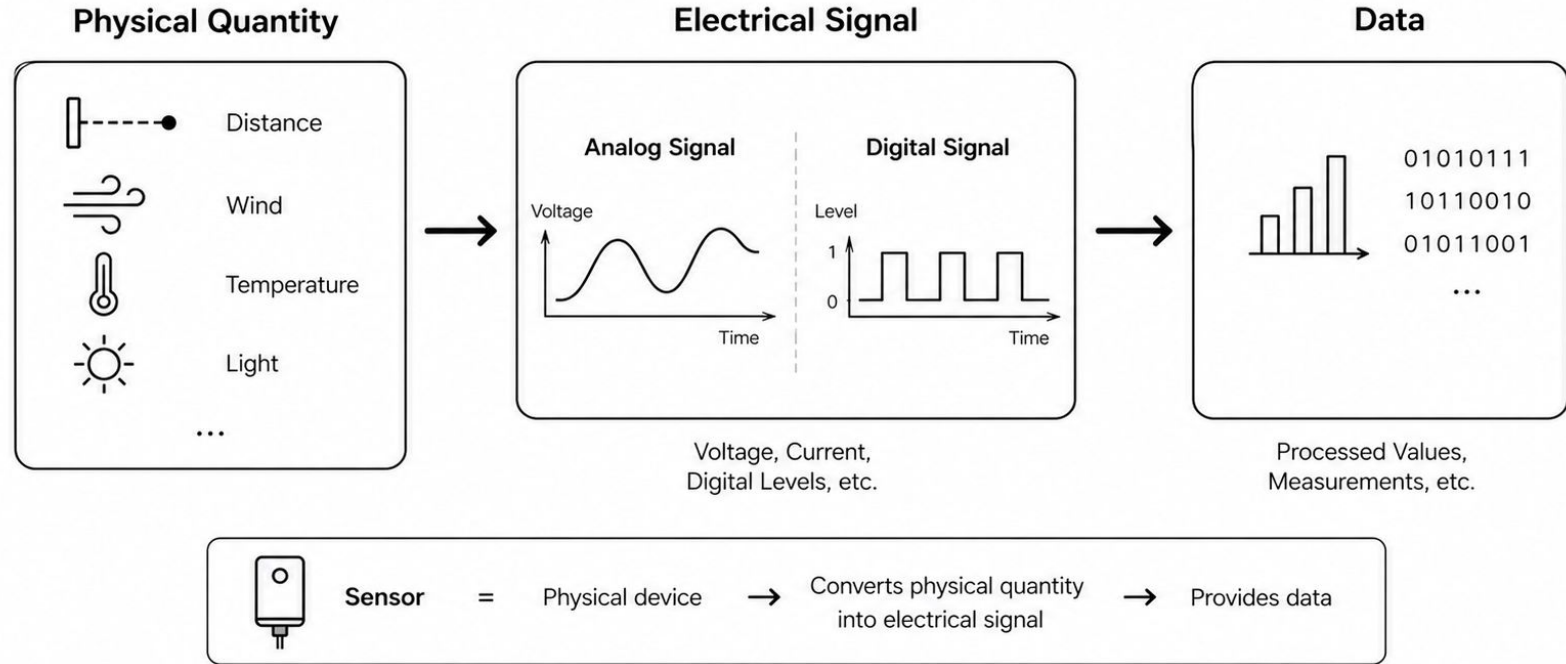
What is a sensor?



What is a sensor?



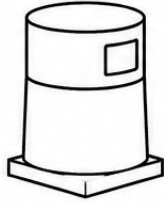
What is a sensor?



From Quantities to Sensors

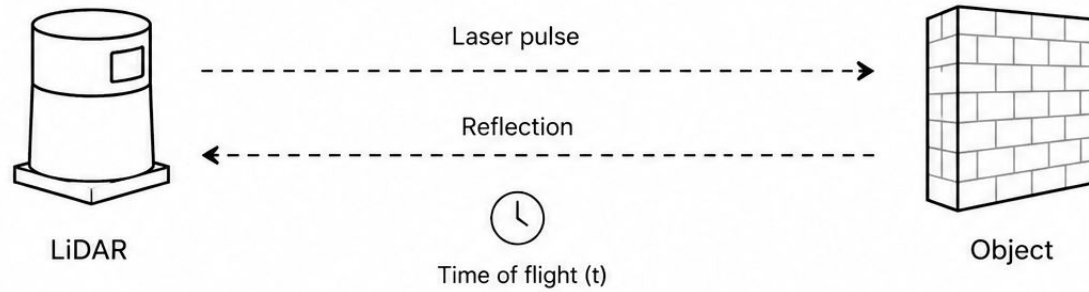
Measured quantity	Sensor
Distance	Lidar / Ultrasonic
Wind	Anemometer
Orientation	IMU
Obstacles	Camera / Lidar

Lidar

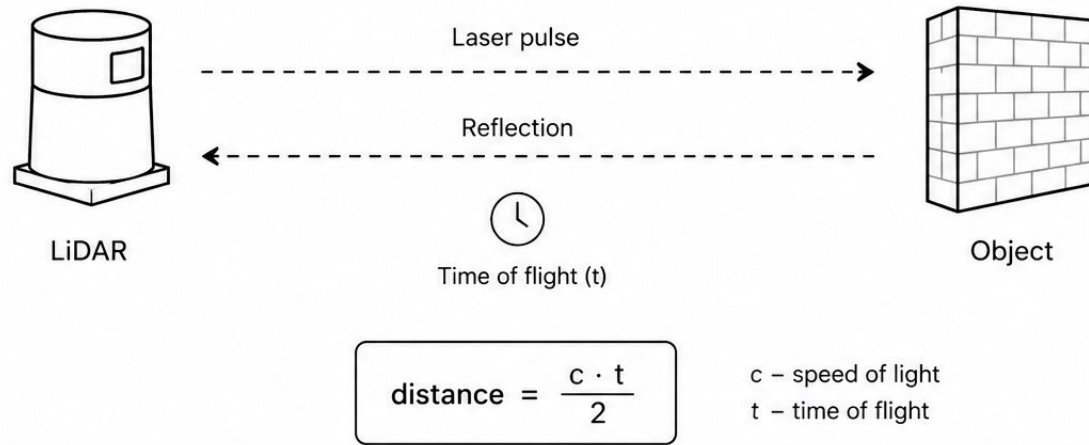


LiDAR

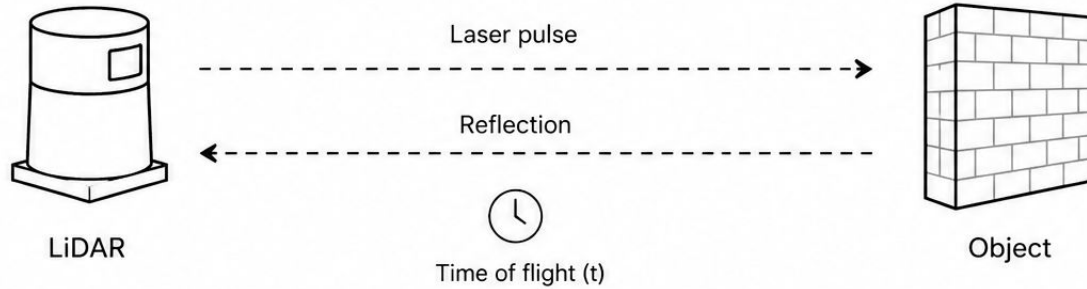
Lidar



Lidar



Lidar



$$\text{distance} = \frac{c \cdot t}{2}$$

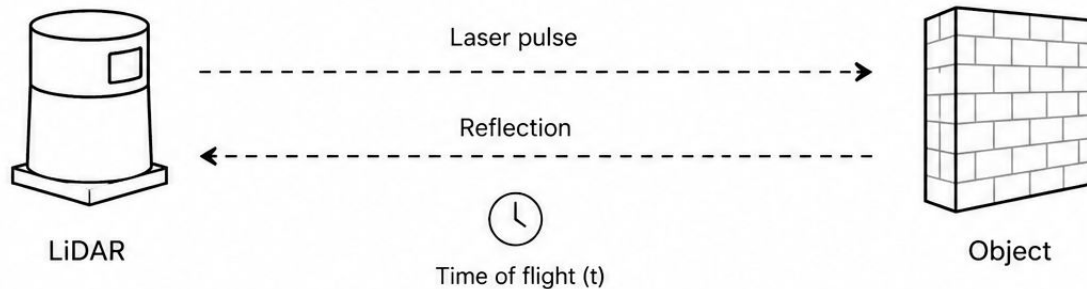
c – speed of light
 t – time of flight

1. Emit Pulse



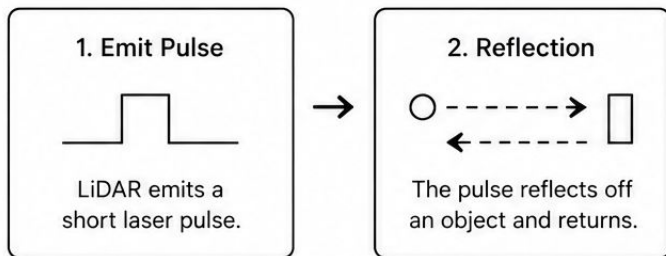
LiDAR emits a short laser pulse.

Lidar

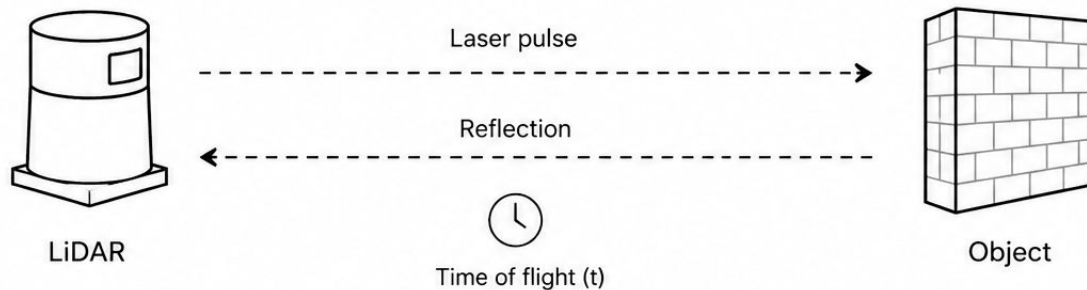


$$\text{distance} = \frac{c \cdot t}{2}$$

c – speed of light
 t – time of flight

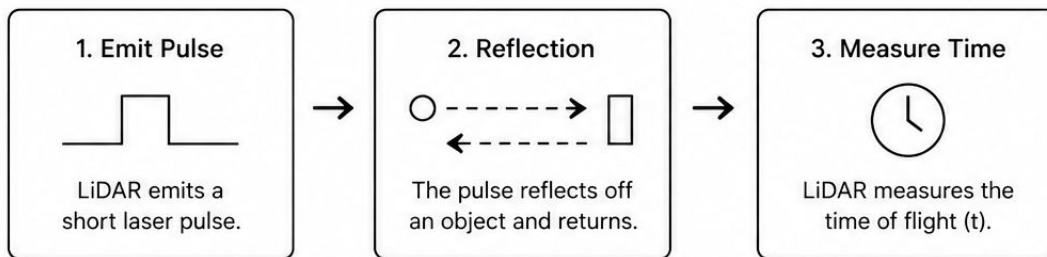


Lidar

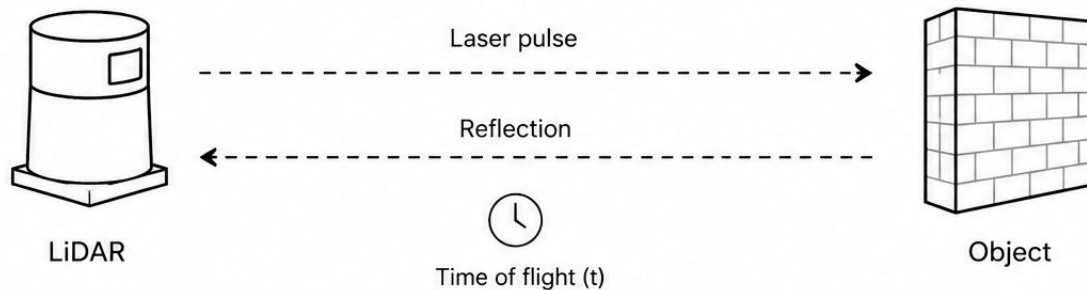


$$\text{distance} = \frac{c \cdot t}{2}$$

c – speed of light
 t – time of flight

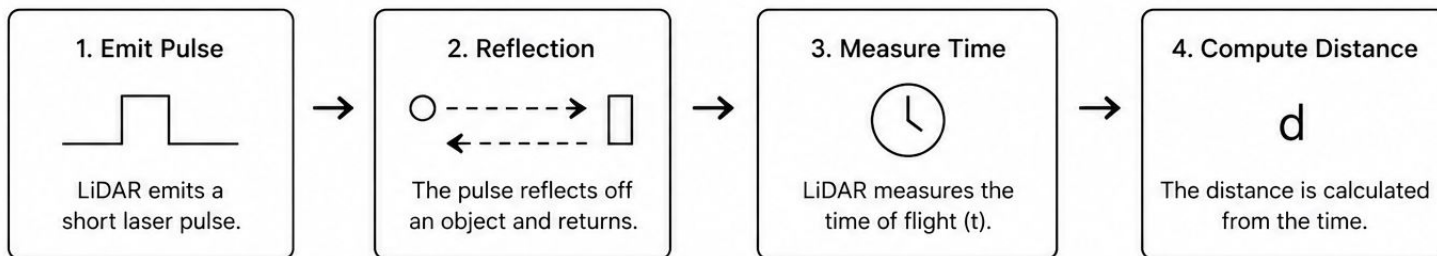


Lidar

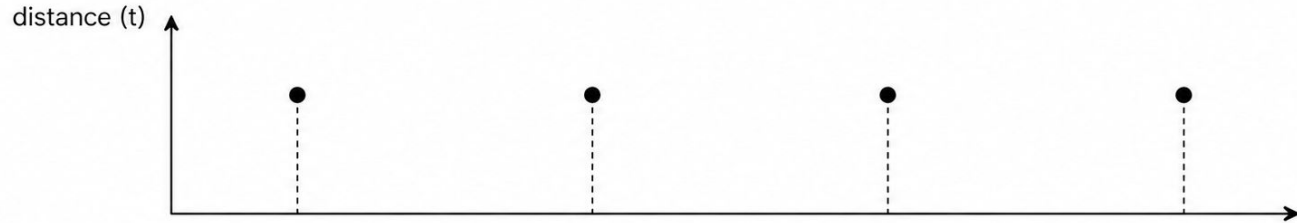


$$\text{distance} = \frac{c \cdot t}{2}$$

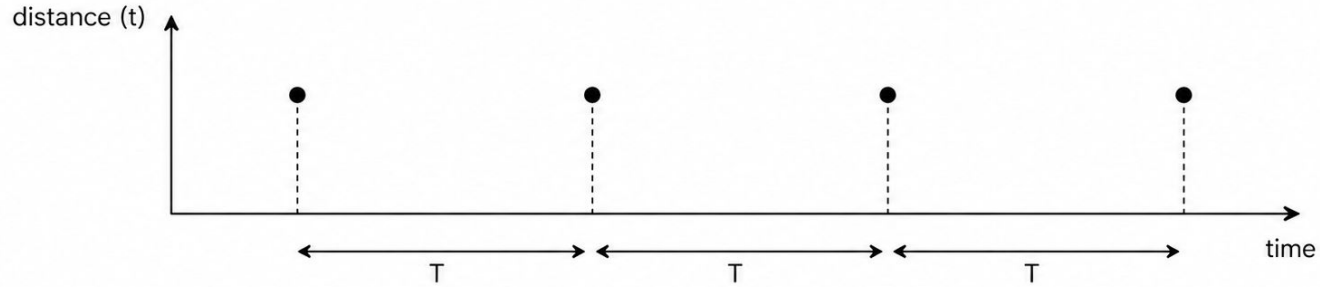
c – speed of light
 t – time of flight



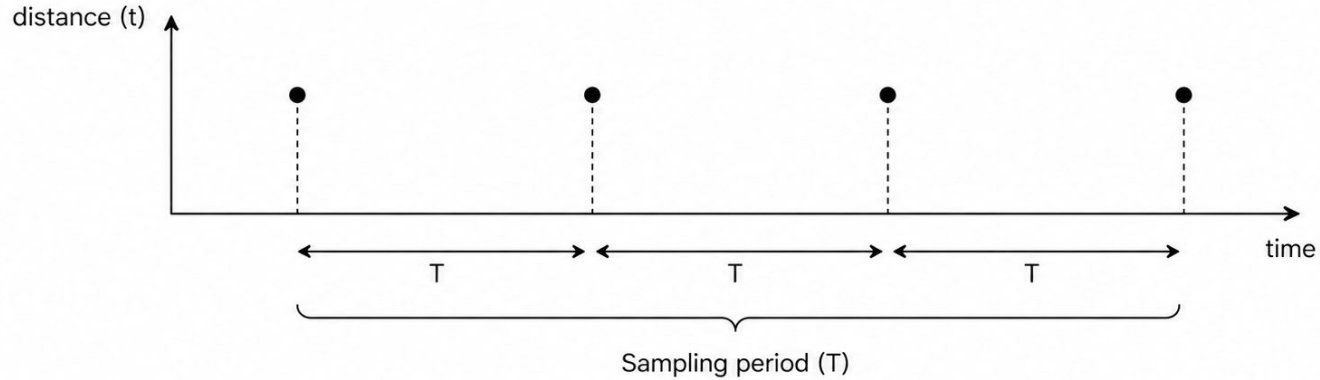
Sensor Output Over Time



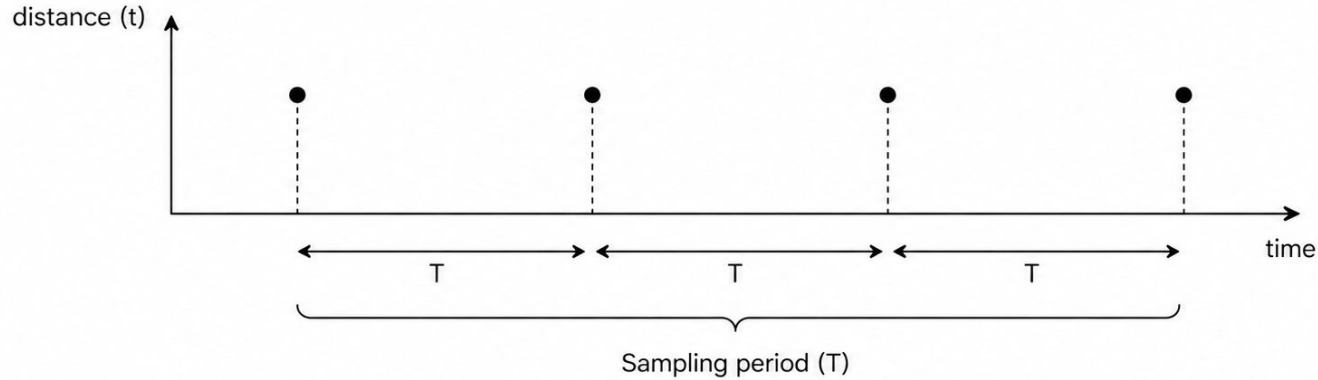
Sensor Output Over Time



Sensor Output Over Time



Sensor Output Over Time

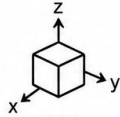


$$f = \frac{1}{T} \quad (\text{sampling frequency})$$

Multiple Sensors



LiDAR

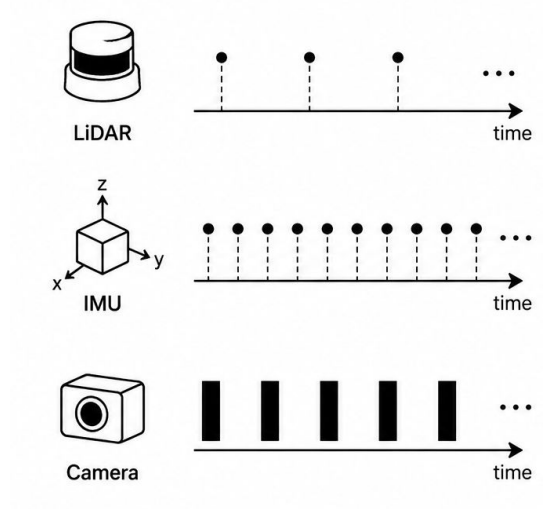


IMU

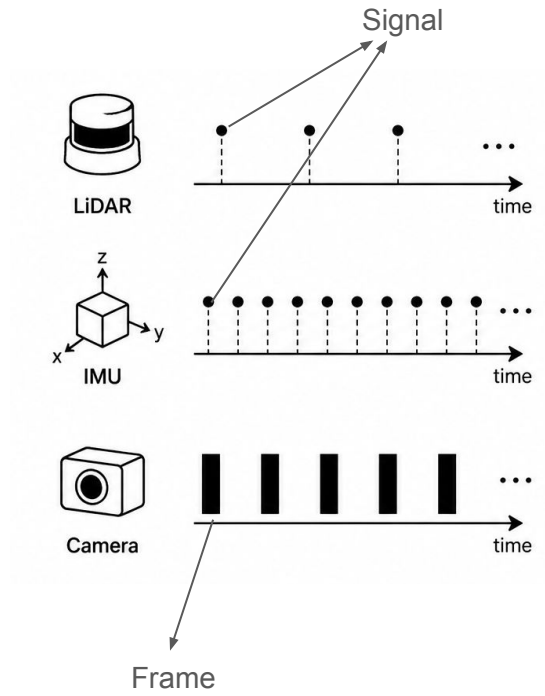


Camera

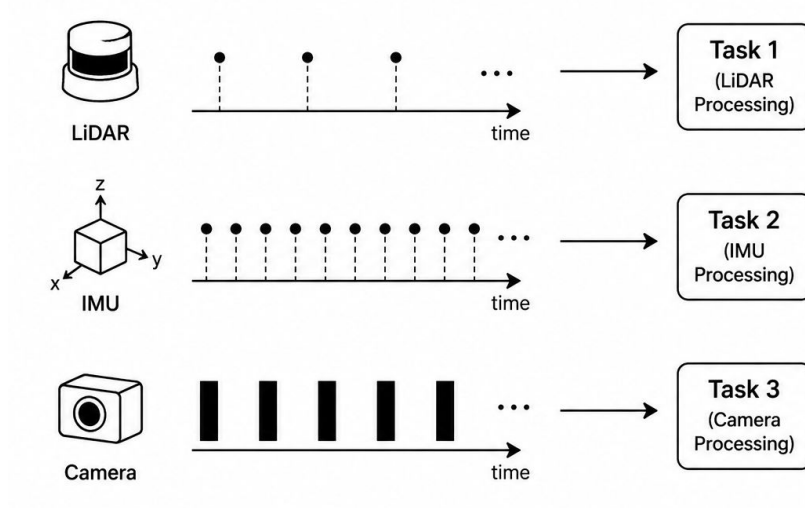
Multiple Sensors



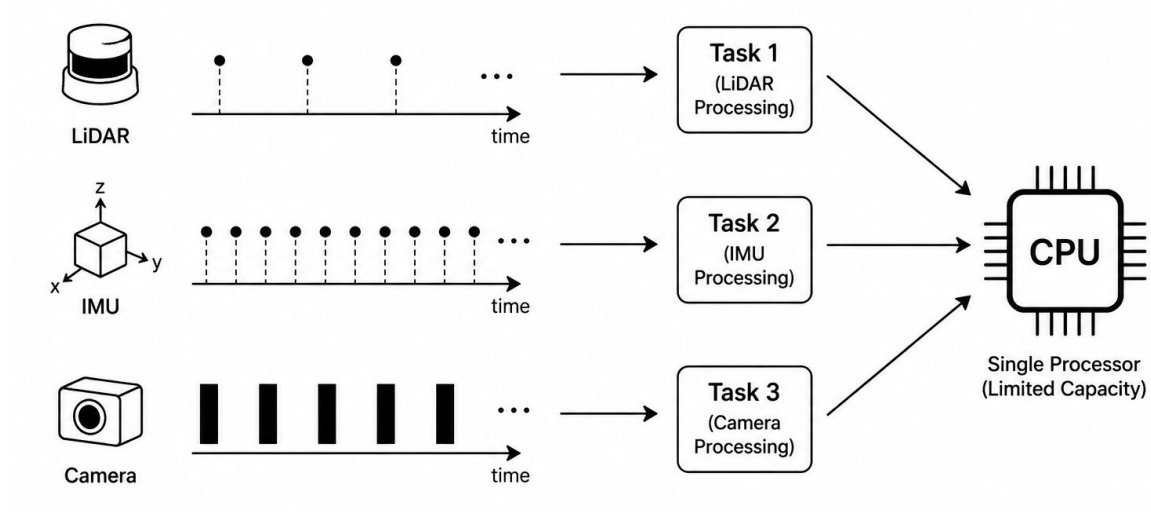
Multiple Sensors



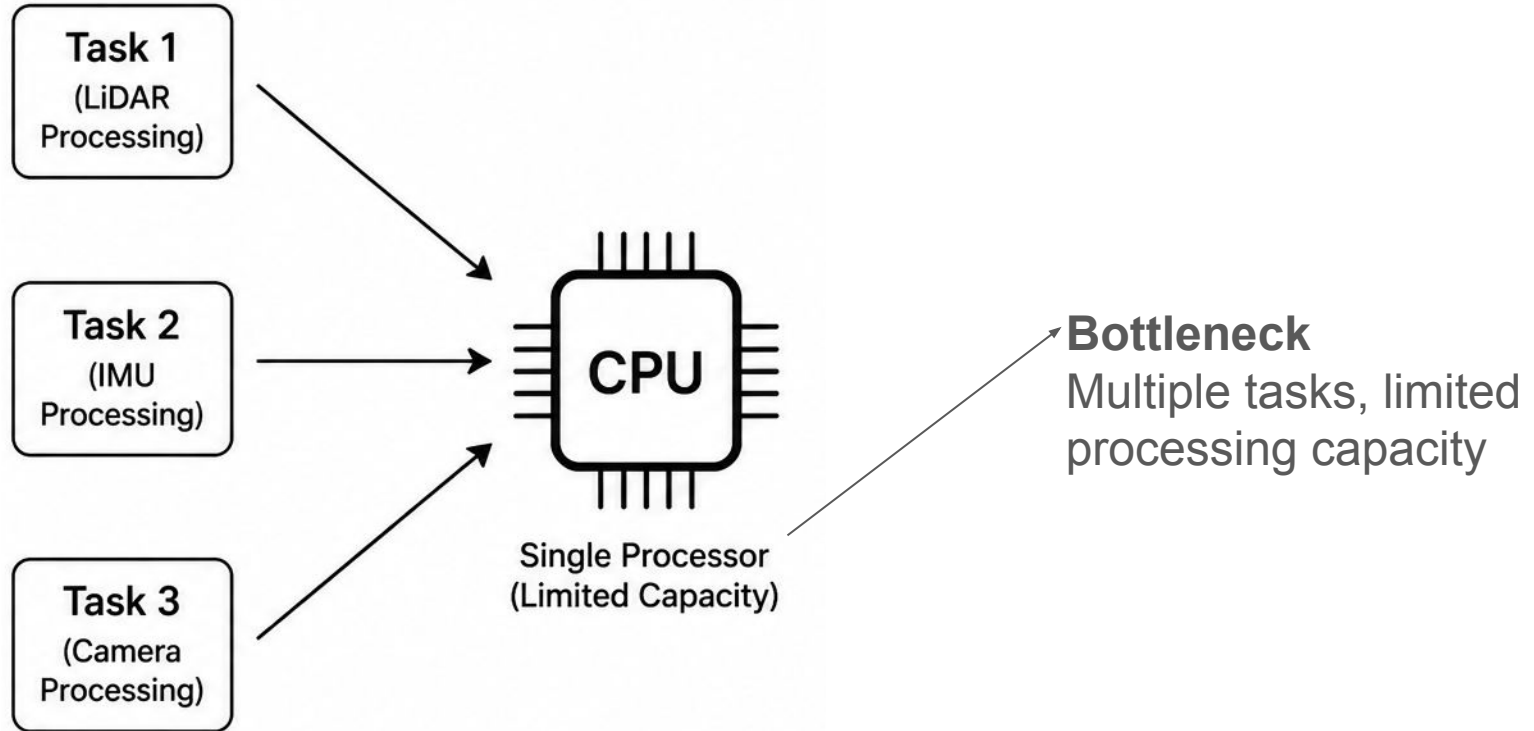
Multiple Sensors



Multiple Sensors



Problem



Need for Scheduling

Set of tasks

Need for Scheduling

Set of tasks  Which task runs first?

Need for Scheduling

Set of tasks → Which task runs first? → CPU

Need for Scheduling

Set of tasks → Which task runs first? → **CPU**

Processor

CPU

Arduino

ESP32

STM32

Processing Units Comparison

	Arduino	ESP32	STM32
--	----------------	--------------	--------------

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Very easy, ready-to-use libraries, minimal setup	Easy, but requires more configuration	Requires setup, low level programming

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Very easy, ready-to-use libraries, minimal setup	Easy, but requires more configuration	Requires setup, low level programming
Performance	Limited performance, suitable for simple tasks	Good performance, dual-core, handles multiple tasks	High performance, optimized for embedded control

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Very easy, ready-to-use libraries, minimal setup	Easy, but requires more configuration	Requires setup, low level programming
Performance	Limited performance, suitable for simple tasks	Good performance, dual-core, handles multiple tasks	High performance, optimized for embedded control
Control	Limited control over hardware and timing	Moderate control, some low-level access	Full control over hardware, precise timing possible

Processing Units Comparison

	Arduino	ESP32	STM32
Ease of use	Very easy, ready-to-use libraries, minimal setup	Easy, but requires more configuration	Requires setup, low level programming
Performance	Limited performance, suitable for simple tasks	Good performance, dual-core, handles multiple tasks	High performance, optimized for embedded control
Control	Limited control over hardware and timing	Moderate control, some low-level access	Full control over hardware, precise timing possible

Why STM32

- Precise control over execution

Why STM32

- Precise control over execution
- Accurate timing measurement

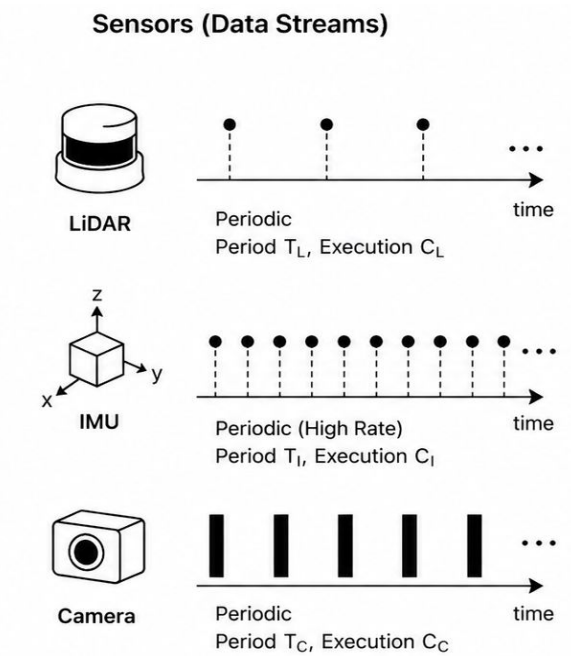
Why STM32

- Precise control over execution
- Accurate timing measurement
- Interrupt-driven behavior

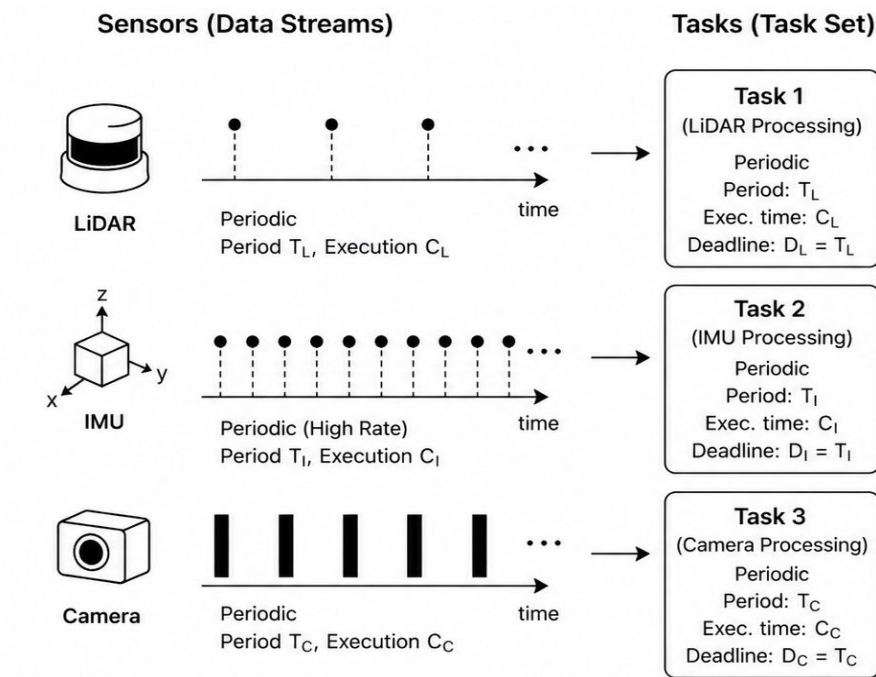
Why STM32

- Precise control over execution
- Accurate timing measurement
- Interrupt-driven behavior
- Low-level access to hardware

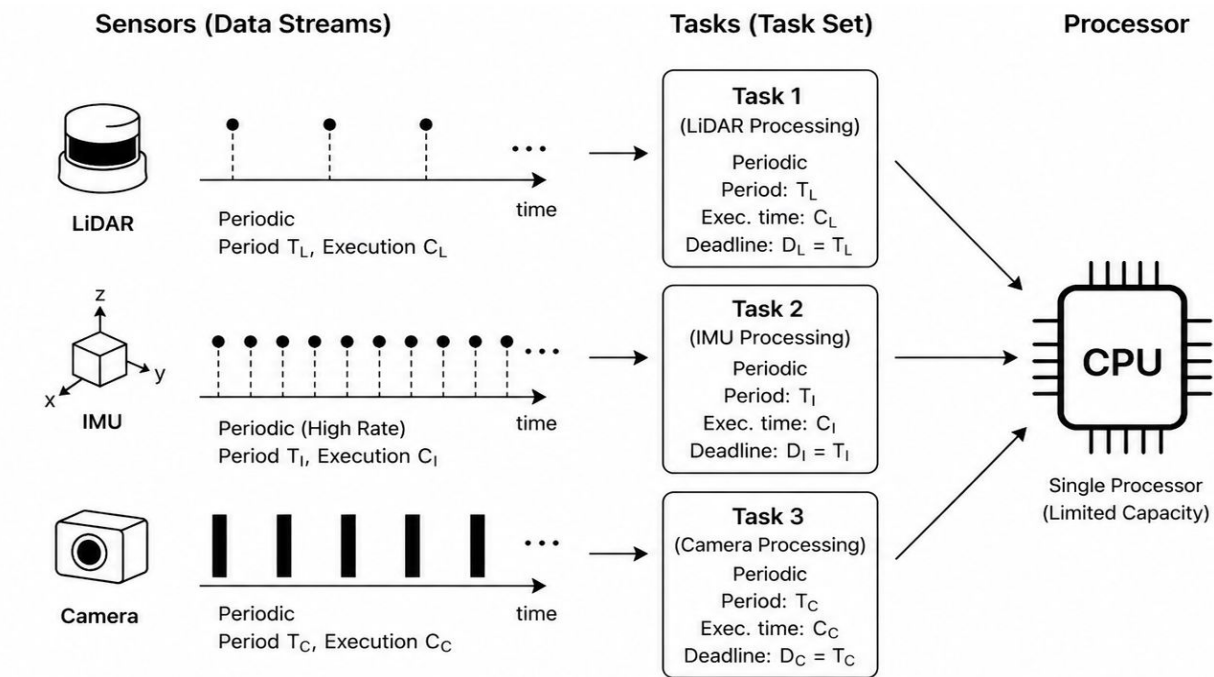
Tasks



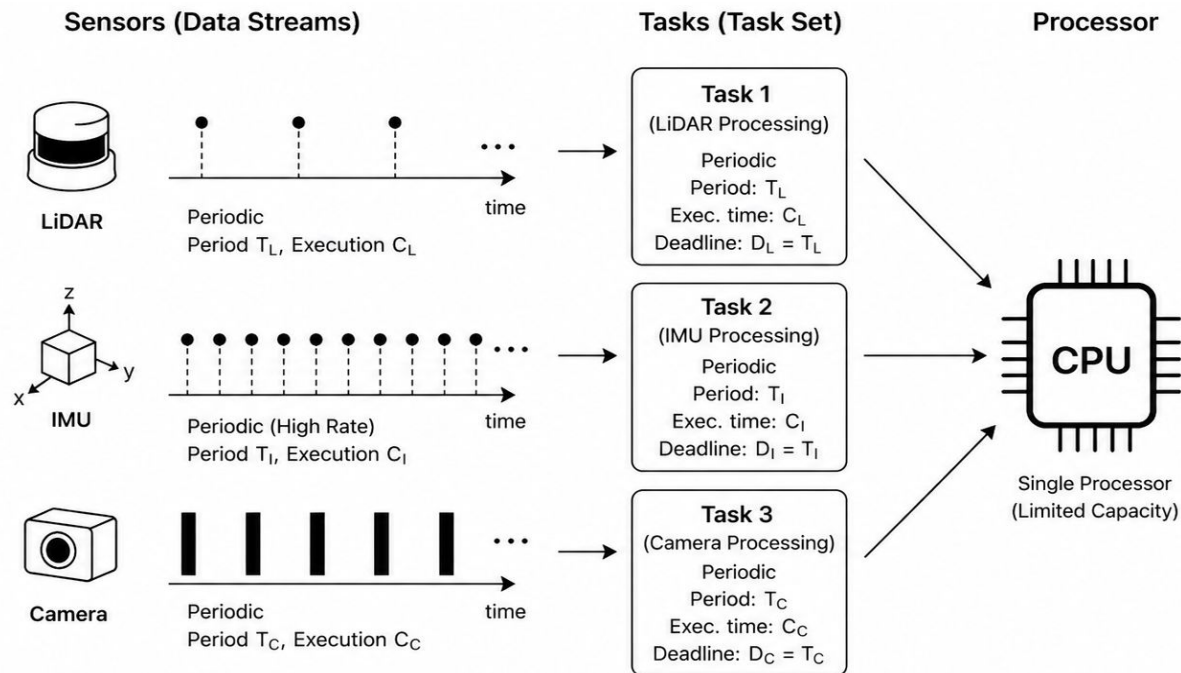
Tasks



Tasks



Tasks



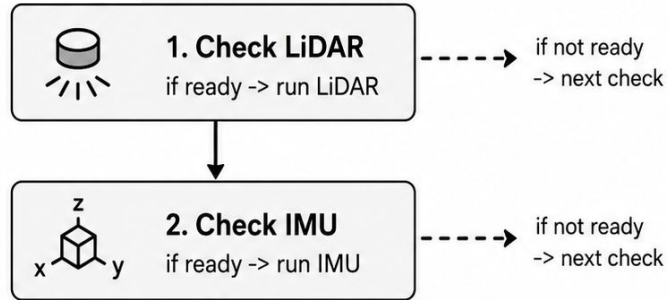
Task Set Summary

Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	T_L (e.g., 100 ms)	C_L (e.g., 20 ms)	$D_L = T_L$	Environment mapping
Task 2	IMU	Periodic (High Rate)	T_I (e.g., 5 ms)	C_I (e.g., 1 ms)	$D_I = T_I$	State estimation / Stabilization
Task 3	Camera	Periodic	T_C (e.g., 200 ms)	C_C (e.g., 40 ms)	$D_C = T_C$	Object detection

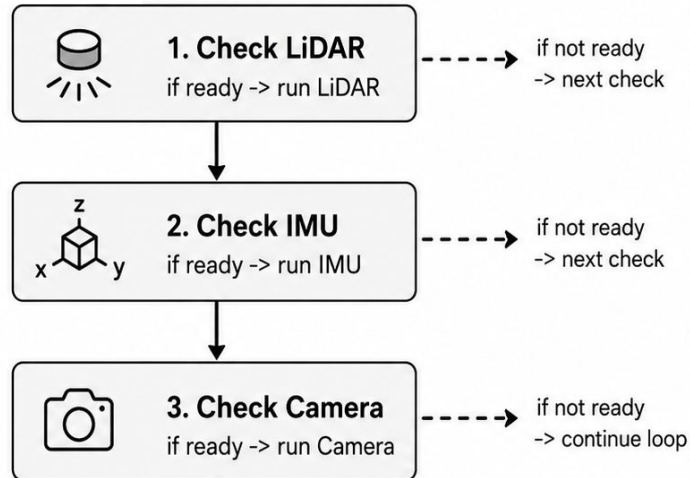
STM32 Execution Logic



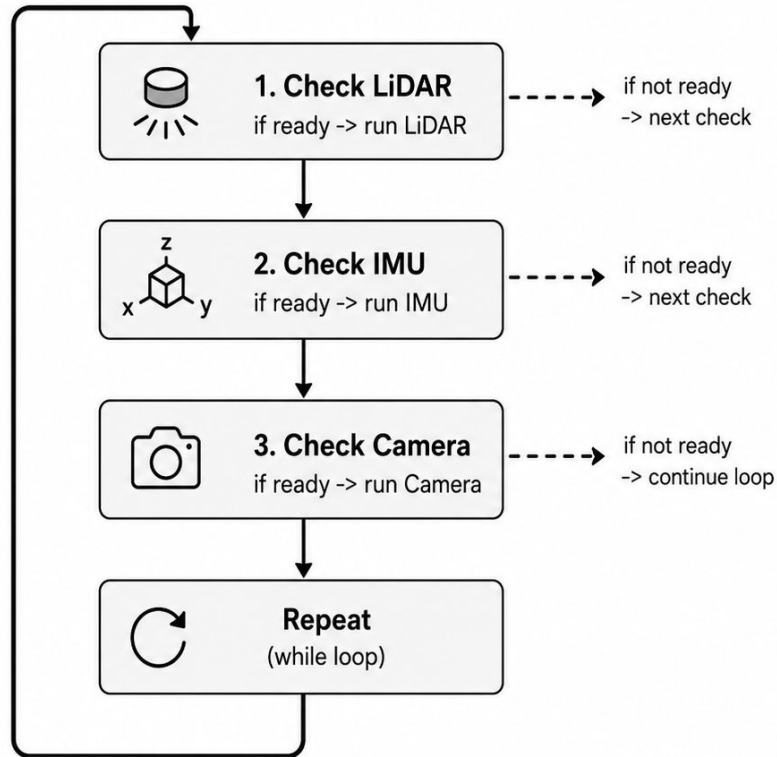
STM32 Execution Logic



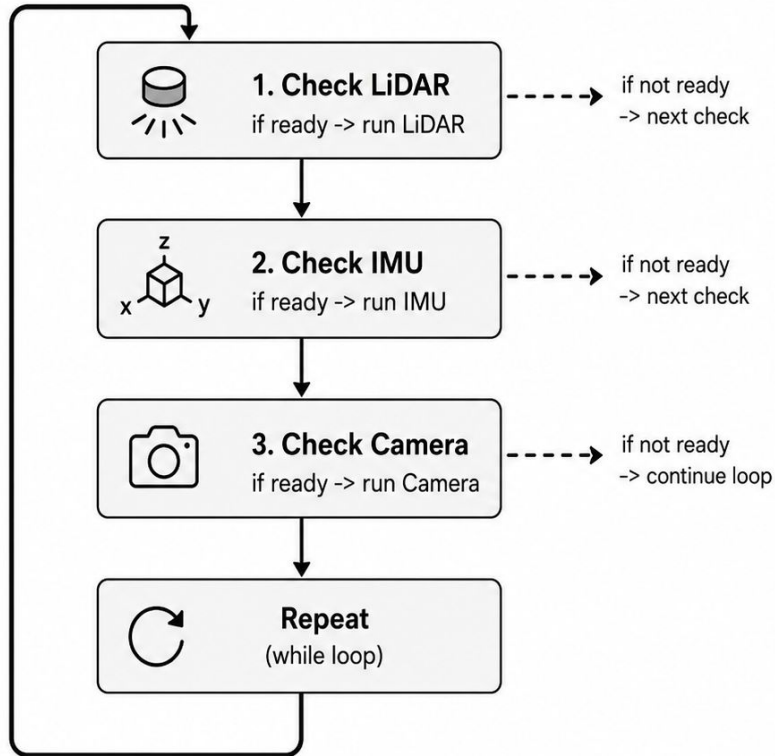
STM32 Execution Logic



STM32 Execution Logic



STM32 Execution Logic

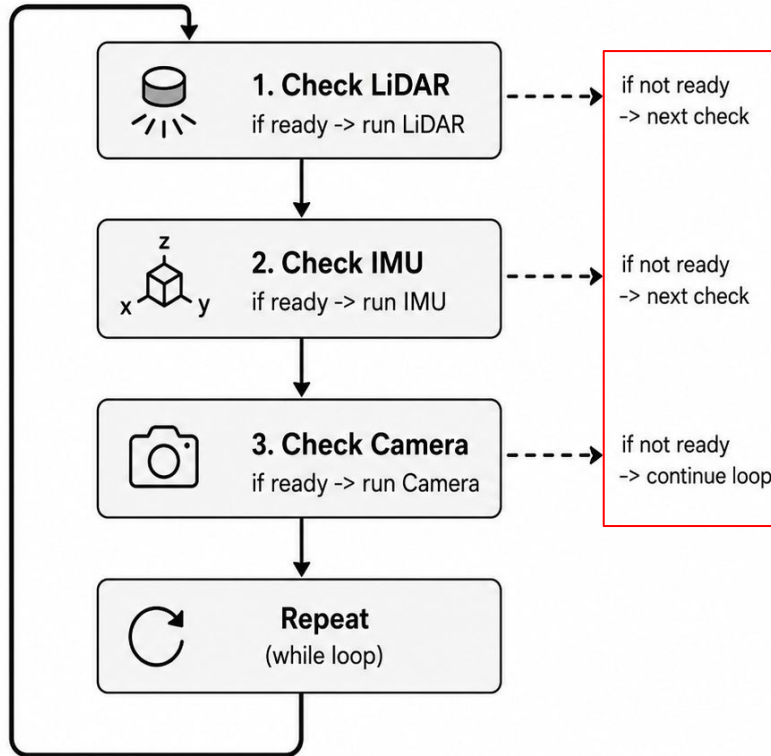


Key Properties

- **Superloop model:**
tasks are checked inside while(1).
- **Sequential code order:**
LiDAR -> IMU -> Camera.
- **Periodic release:**
each task becomes ready according to its own period.
- **Non-preemptive:**
a running task continues until completion.
- **One task at a time:**
only the current task uses the CPU.
- **Skip if not ready:**
the loop moves to the next check.

Note: If multiple tasks are ready during the same iteration, they run in the order defined by the code.

STM32 Execution Logic



Key Properties

- **Superloop model:**
tasks are checked inside while(1).
- **Sequential code order:**
LiDAR -> IMU -> Camera.
- **Periodic release:**
each task becomes ready according to its own period.
- **Non-preemptive:**
a running task continues until completion.
- **One task at a time:**
only the current task uses the CPU.
- **Skip if not ready:**
the loop moves to the next check.

Note: If multiple tasks are ready during the same iteration, they run in the order defined by the code.

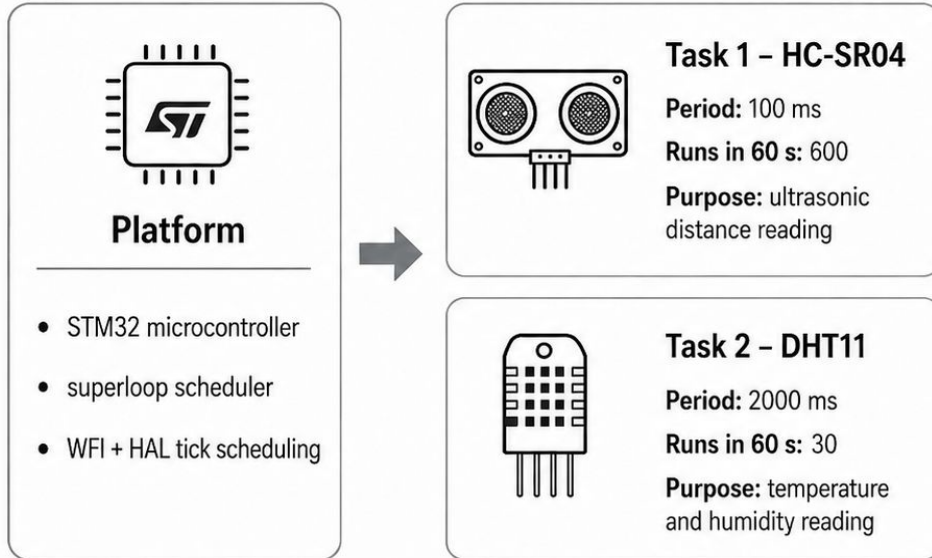
Profiling: Cost of Sequential Checks



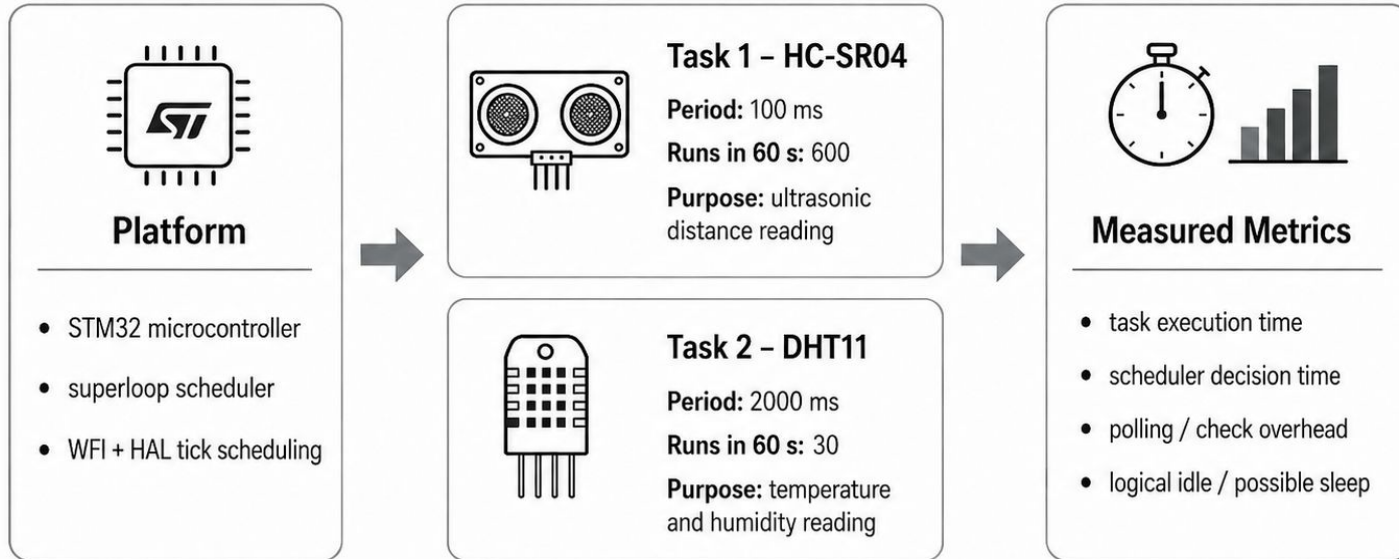
Platform

- STM32 microcontroller
- superloop scheduler
- WFI + HAL tick scheduling

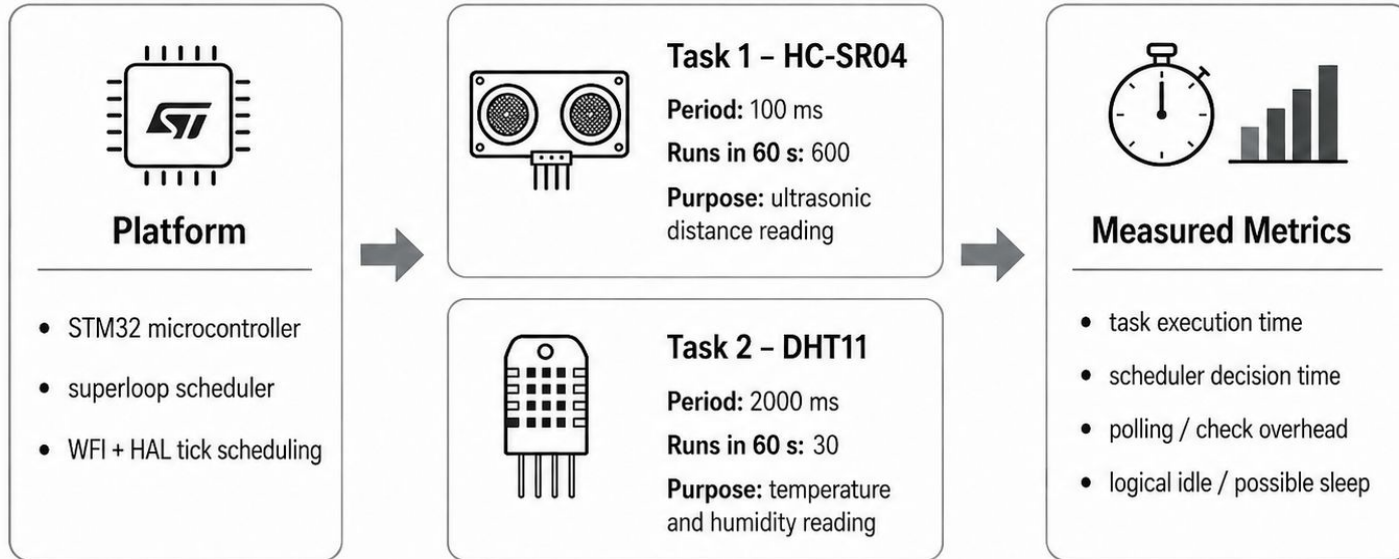
Profiling: Cost of Sequential Checks



Profiling: Cost of Sequential Checks



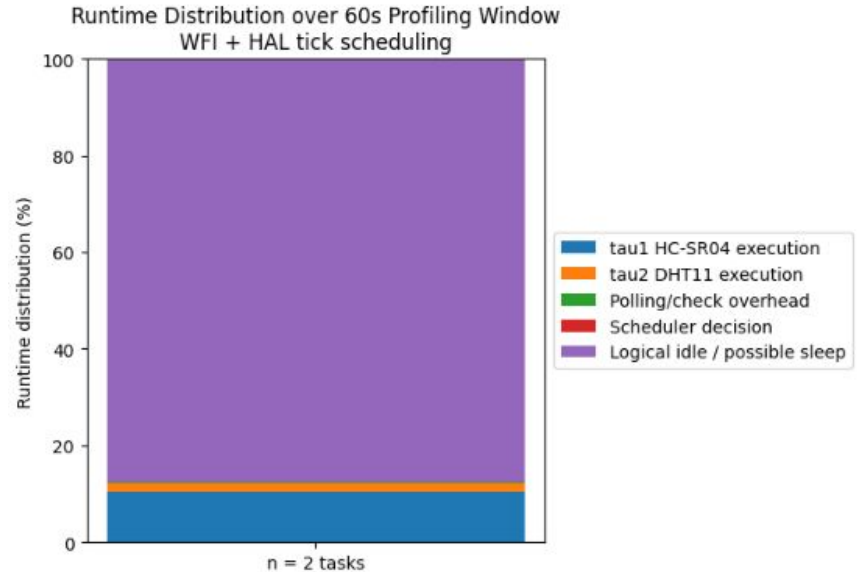
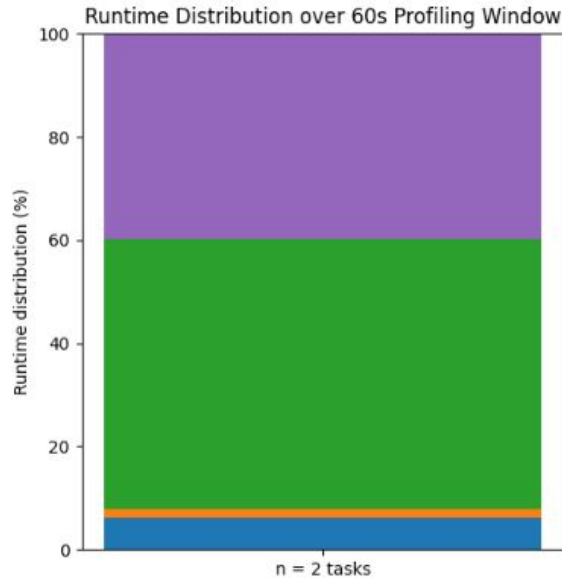
Profiling: Cost of Sequential Checks



Goal

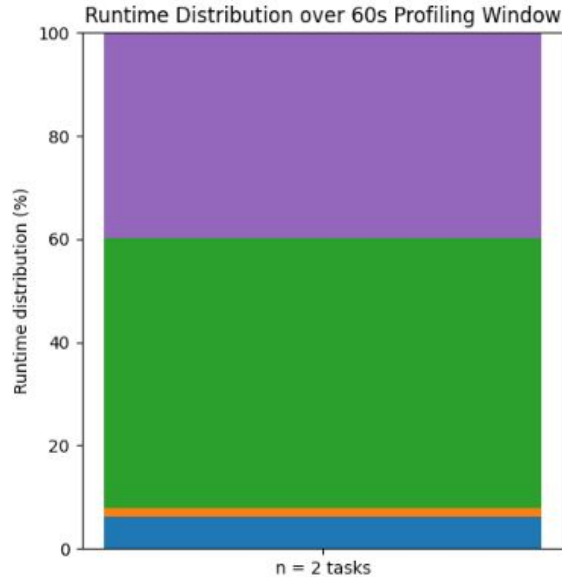
Show that the cost of sequential checks is small after optimization, so the main issue becomes scheduling and task ordering.

Profiling: Results

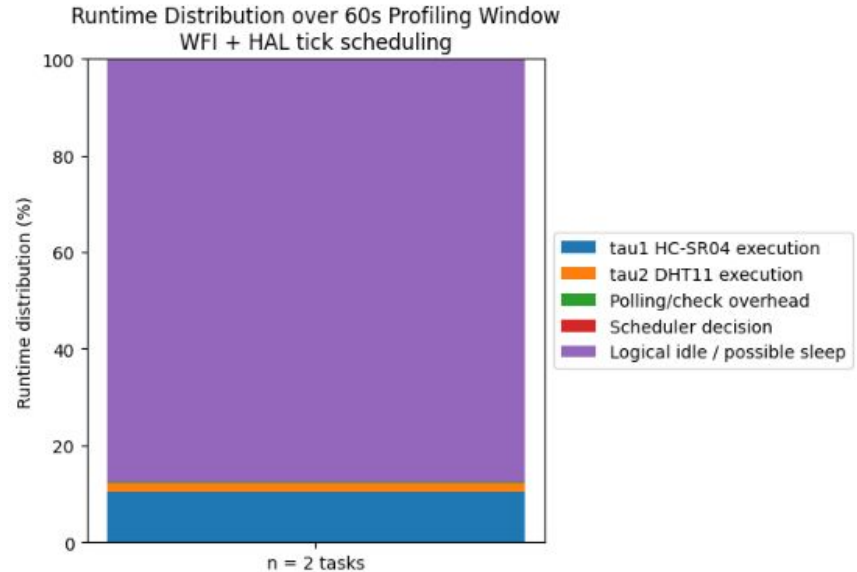


Profiling: Results

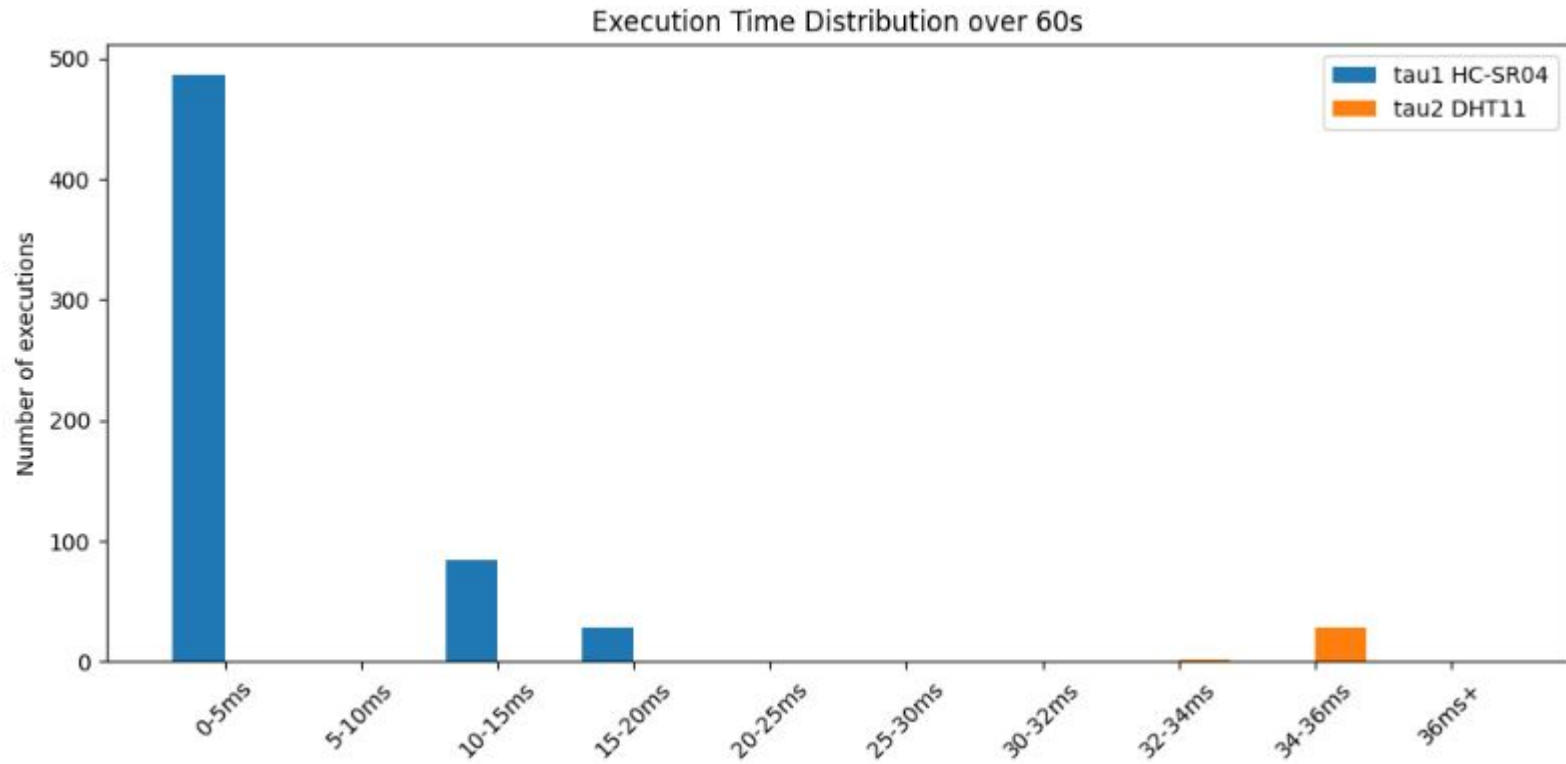
No optimization



With optimization



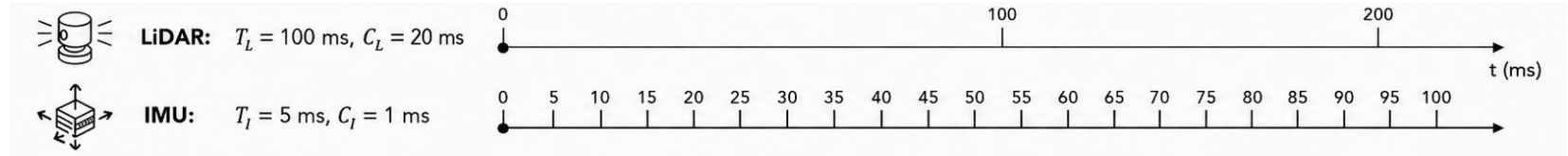
Execution time



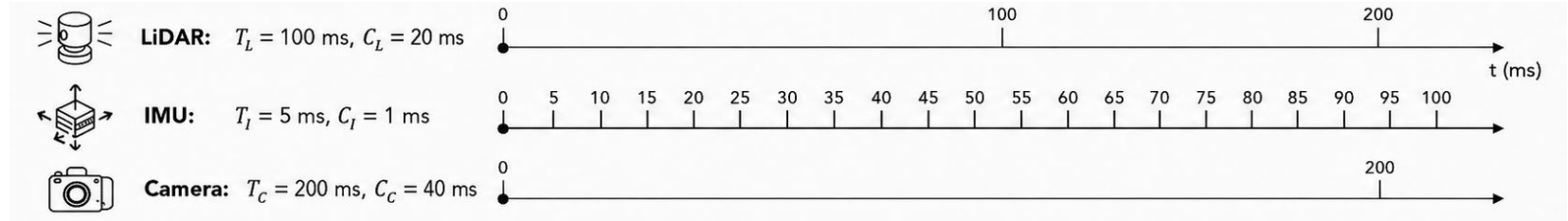
CPU Execution Timeline



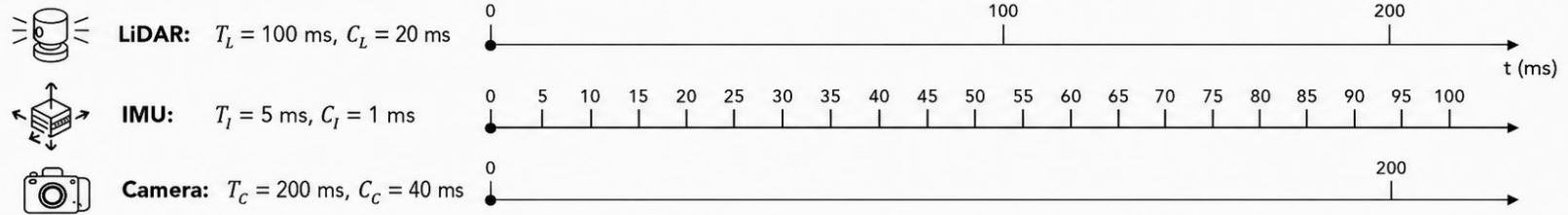
CPU Execution Timeline



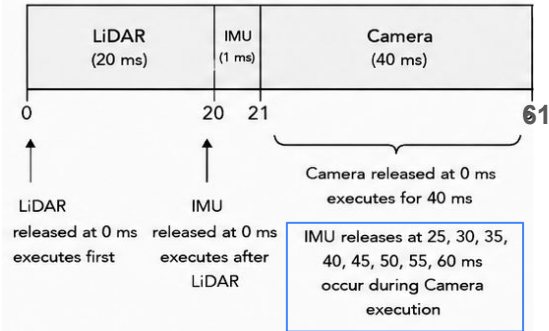
CPU Execution Timeline



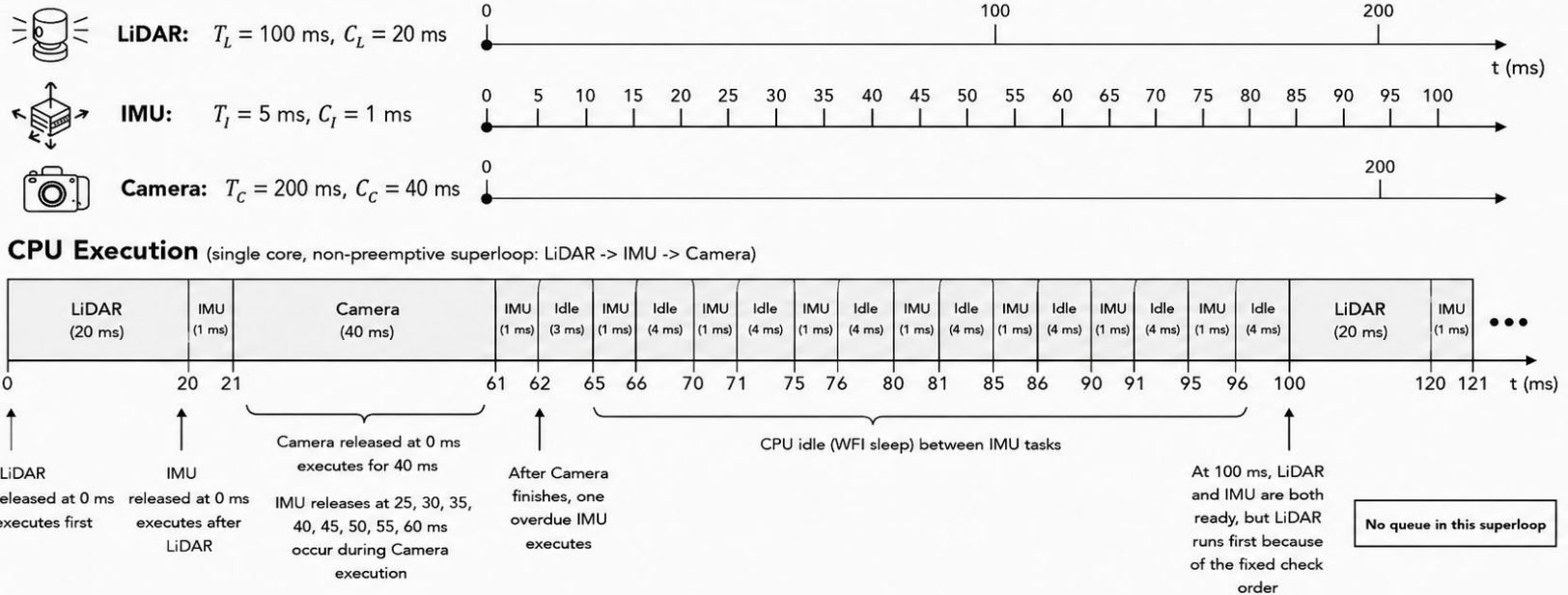
CPU Execution Timeline



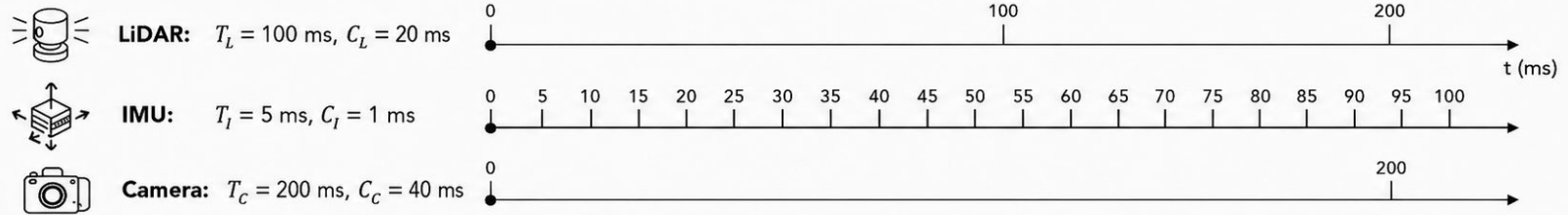
CPU Execution (single core, non-preemptive superloop: LiDAR -> IMU -> Camera)



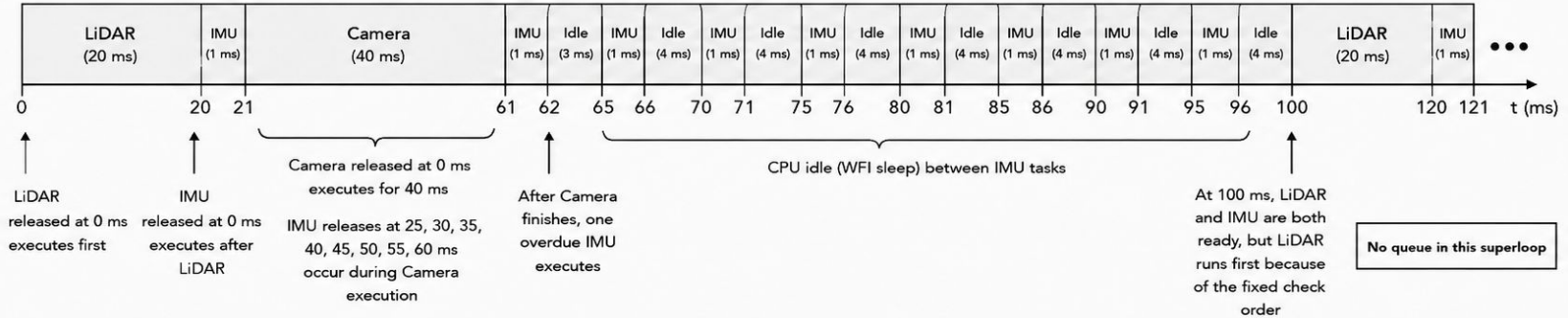
CPU Execution Timeline



CPU Execution Timeline



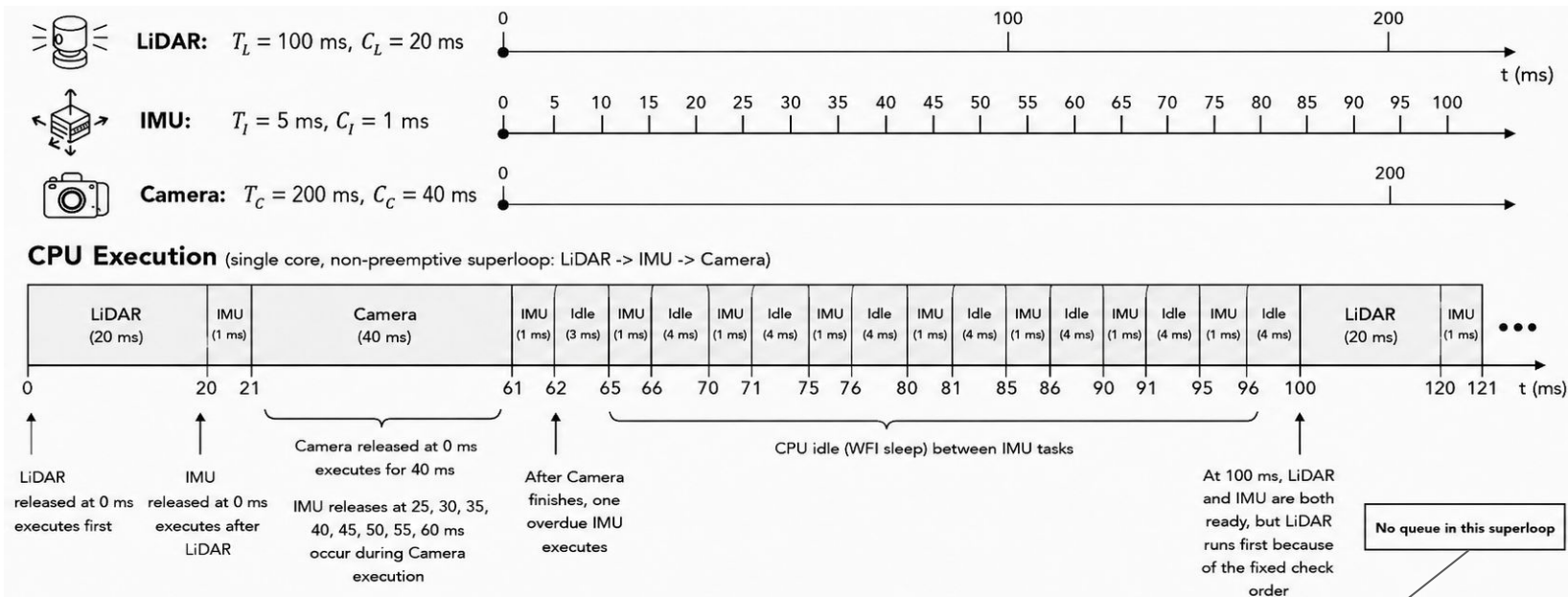
CPU Execution (single core, non-preemptive superloop: LiDAR -> IMU -> Camera)



Key points

- Only one task executes at a time.
- The check order is fixed: LiDAR -> IMU -> Camera.
- The system is non-preemptive.
- Releases can occur while another task is running.

CPU Execution Timeline



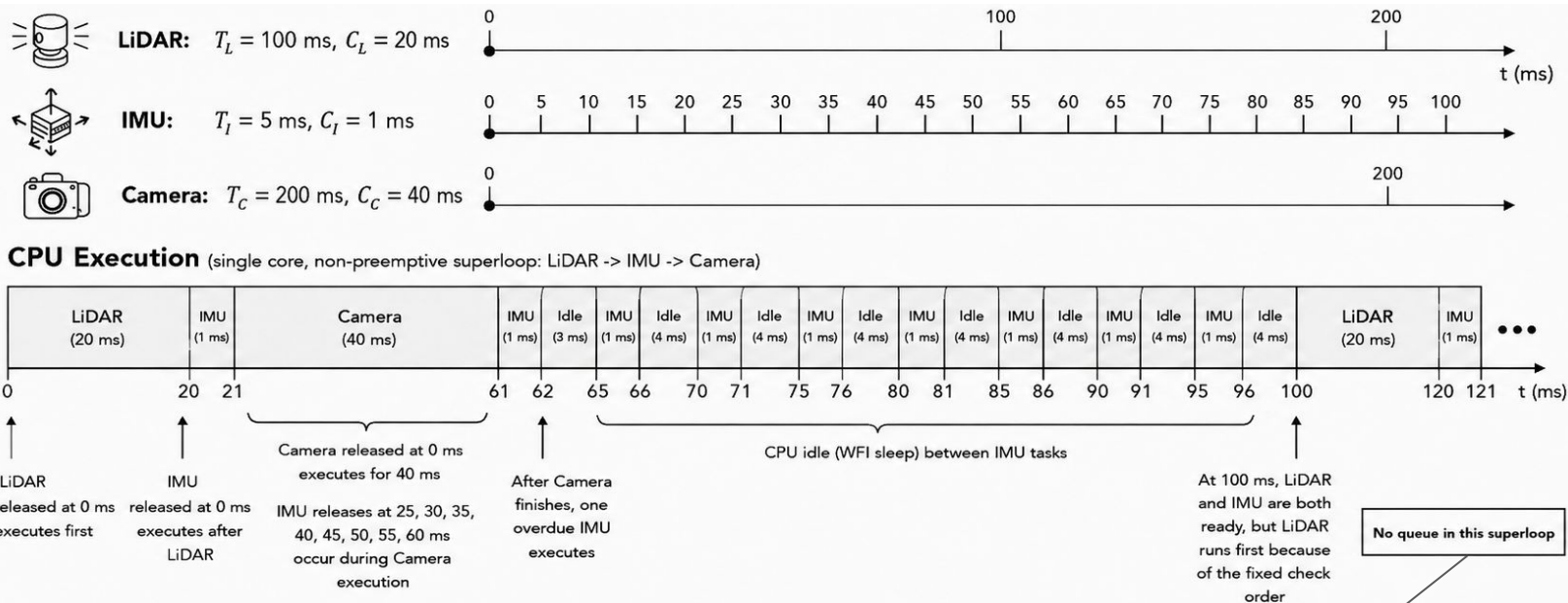
Key points

- Only one task executes at a time.
- The check order is fixed: LiDAR -> IMU -> Camera.
- The system is non-preemptive.
- Releases can occur while another task is running.

What this means

- Missed IMU releases do not accumulate as a queue.
- Multiple missed IMU releases collapse into one overdue handling point.
- Long tasks can delay fast sensor tasks.
- When LiDAR and IMU are both ready, LiDAR runs first because it is checked first.

CPU Execution Timeline



Problem

Key points

- Only one task executes at a time.
- The check order is fixed: LiDAR -> IMU -> Camera.
- The system is non-preemptive.
- Releases can occur while another task is running.

What this means

- Missed IMU releases do not accumulate as a queue.
- Multiple missed IMU releases collapse into one overdue handling point.
- Long tasks can delay fast sensor tasks.
- When LiDAR and IMU are both ready, LiDAR runs first because it is checked first.

Why is this a problem?

- High-frequency tasks are delayed

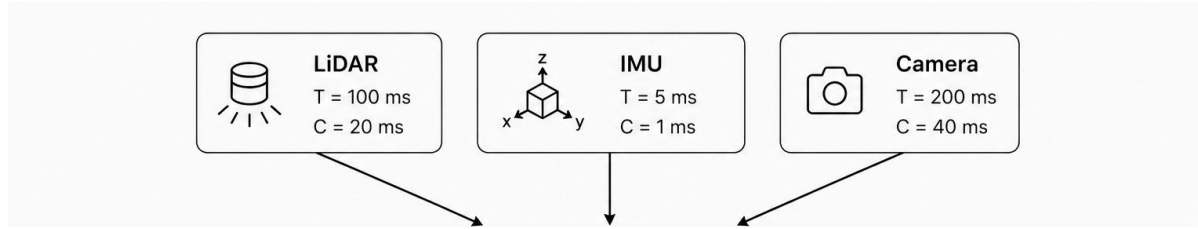
Why is this a problem?

- High-frequency tasks are delayed
- Long tasks block execution

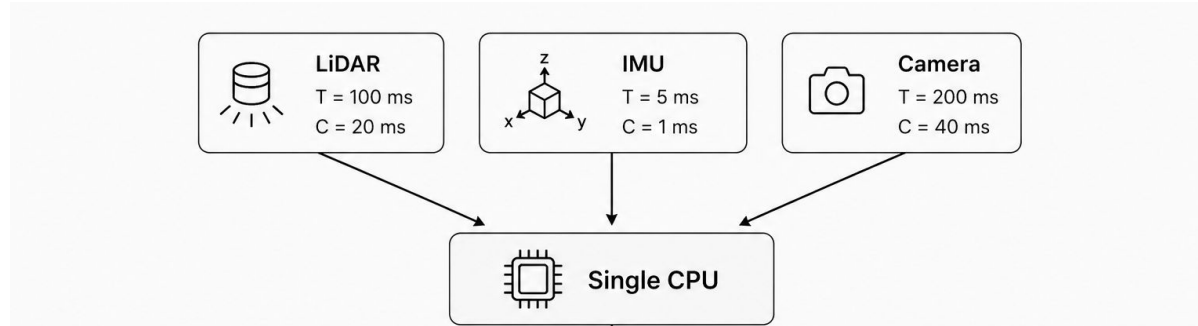
Why is this a problem?

- High-frequency tasks are delayed
- Long tasks block execution
- Timing is limited by code order

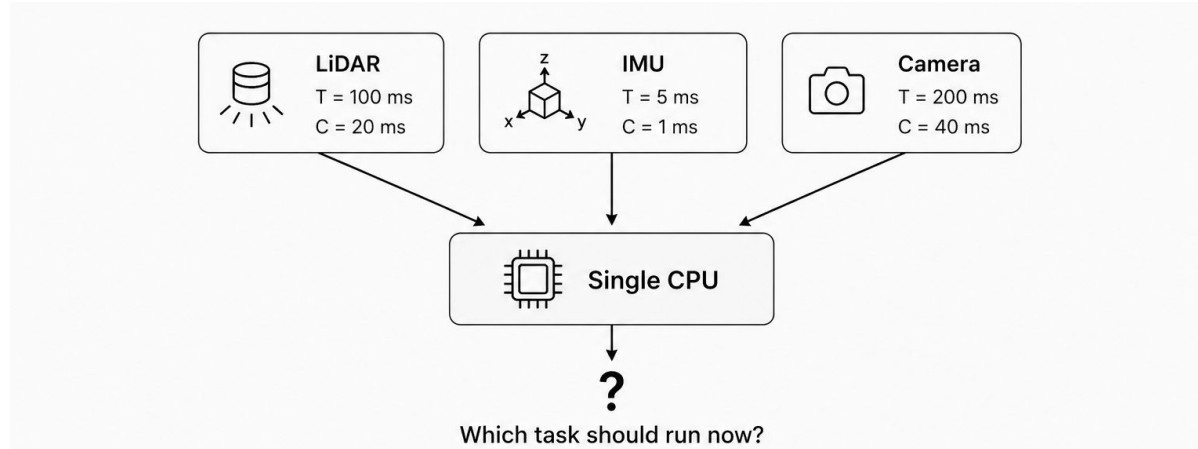
Scheduling Problem



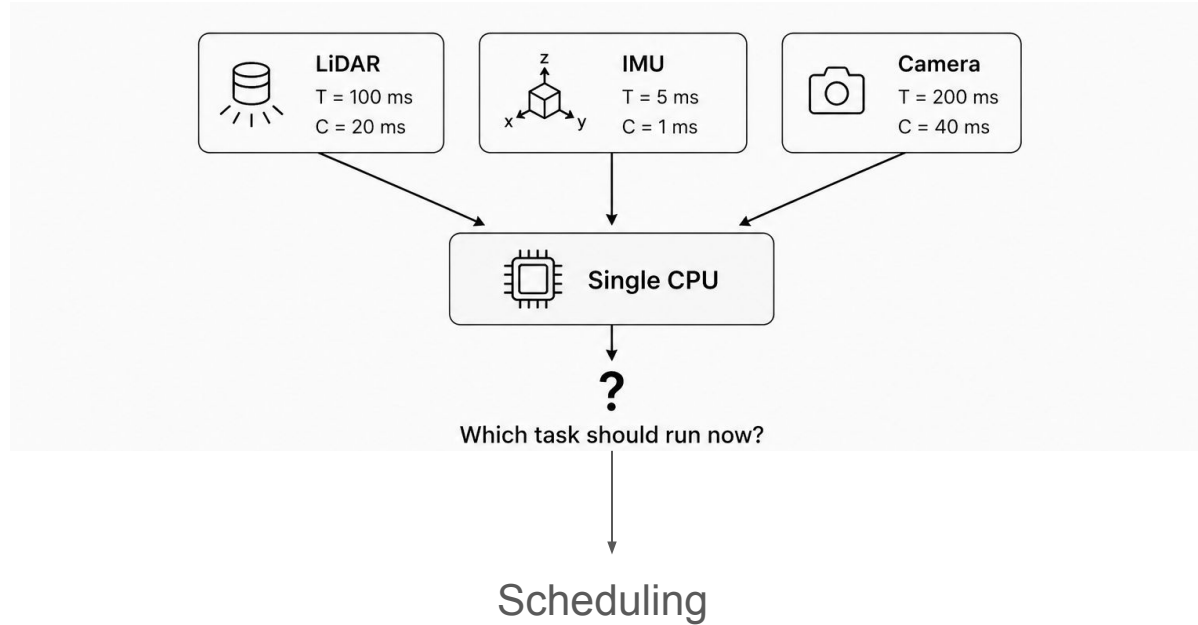
Scheduling Problem



Scheduling Problem



Scheduling



Earliest Deadline First (EDF)

Task	Period	Relative deadline
------	--------	-------------------

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	5 ms	5 ms

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	5 ms	5 ms
LiDAR	100 ms	100 ms

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	5 ms	5 ms
LiDAR	100 ms	100 ms
Camera	200 ms	200 ms

Earliest Deadline First (EDF)

Task	Period	Relative deadline
IMU	5 ms	5 ms
LiDAR	100 ms	100 ms
Camera	200 ms	200 ms

EDF Execution Timeline

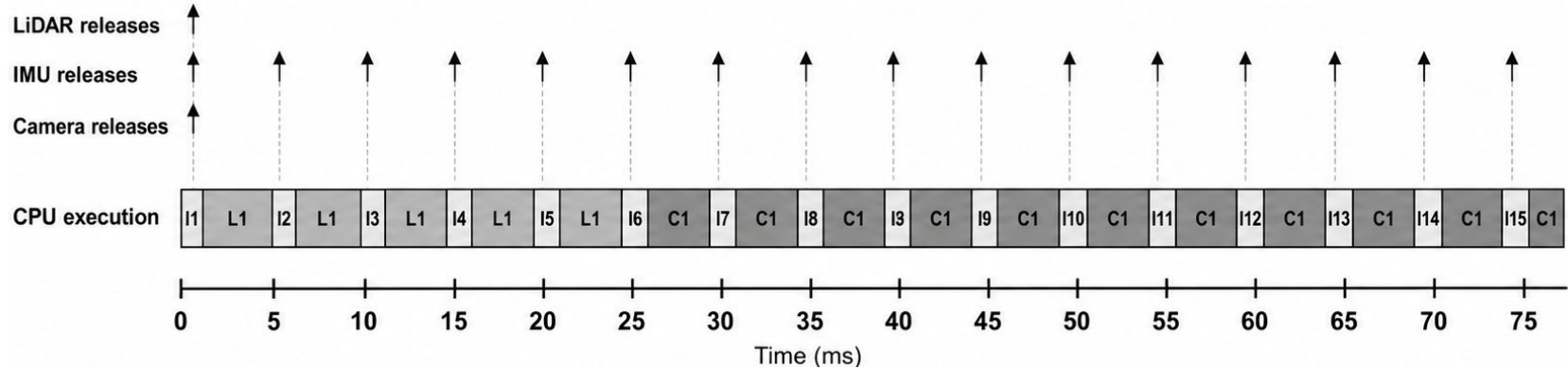
Task Set Summary

Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	$T_L = 100 \text{ ms}$	$C_L = 20 \text{ ms}$	$D_L = T_L = 100 \text{ ms}$	Environment mapping
Task 2	IMU	Periodic (High Rate)	$T_I = 5 \text{ ms}$	$C_I = 1 \text{ ms}$	$D_I = T_I = 5 \text{ ms}$	State estimation / stabilization
Task 3	Camera	Periodic	$T_C = 200 \text{ ms}$	$C_C = 40 \text{ ms}$	$D_C = T_C = 200 \text{ ms}$	Object detection

EDF Execution Timeline

Task Set Summary

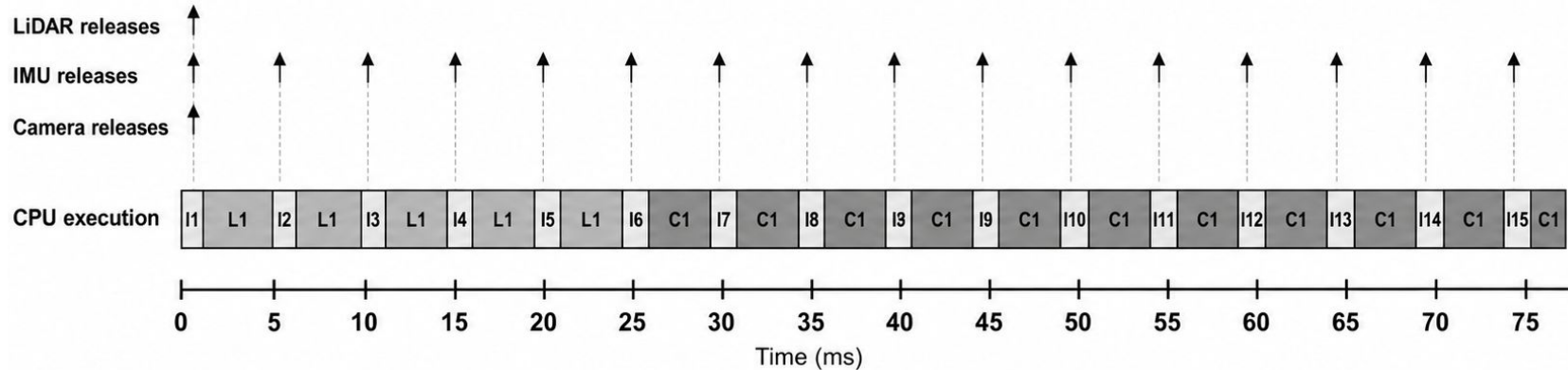
Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	$T_L = 100$ ms	$C_L = 20$ ms	$D_L = T_L = 100$ ms	Environment mapping
Task 2	IMU	Periodic (High Rate)	$T_I = 5$ ms	$C_I = 1$ ms	$D_I = T_I = 5$ ms	State estimation / stabilization
Task 3	Camera	Periodic	$T_C = 200$ ms	$C_C = 40$ ms	$D_C = T_C = 200$ ms	Object detection



EDF Execution Timeline

Task Set Summary

Task	Source	Type	Period (T)	Exec. Time (C)	Deadline (D)	Notes
Task 1	LiDAR	Periodic	$T_L = 100$ ms	$C_L = 20$ ms	$D_L = T_L = 100$ ms	Environment mapping
Task 2	IMU	Periodic (High Rate)	$T_I = 5$ ms	$C_I = 1$ ms	$D_I = T_I = 5$ ms	State estimation / stabilization
Task 3	Camera	Periodic	$T_C = 200$ ms	$C_C = 40$ ms	$D_C = T_C = 200$ ms	Object detection



- EDF always runs the ready job with the earliest absolute deadline.
- IMU preempts LiDAR and Camera every 5 ms because each IMU job has the closest deadline.
- LiDAR (L1) completes at 25 ms after receiving 20 ms total execution.
- Camera (C1) starts at 26 ms and completes at 75 ms after 40 ms total execution.

Limitation of STM32 Execution Model

Aspect	EDF	STM32
--------	-----	-------

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion

Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion
Timing guarantees	Strong guarantees	Hard to guarantee

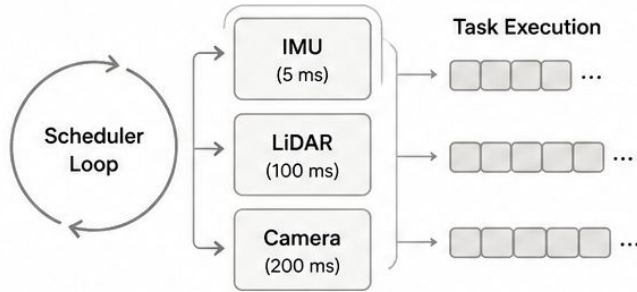
Limitation of STM32 Execution Model

Aspect	EDF	STM32
Scheduling	Preemptive	Non-preemptive
Selection rule	Dynamic deadline-based	Fixed code check order
Task execution	Can be interrupted anytime	Runs to completion
Timing guarantees	Strong guarantees	Hard to guarantee

Design Choice: Implementing Preemption on STM32

1. Task Splitting (Chunks)

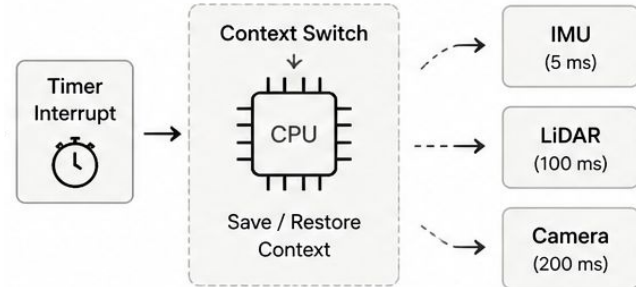
Break tasks into small execution steps
and schedule cooperatively



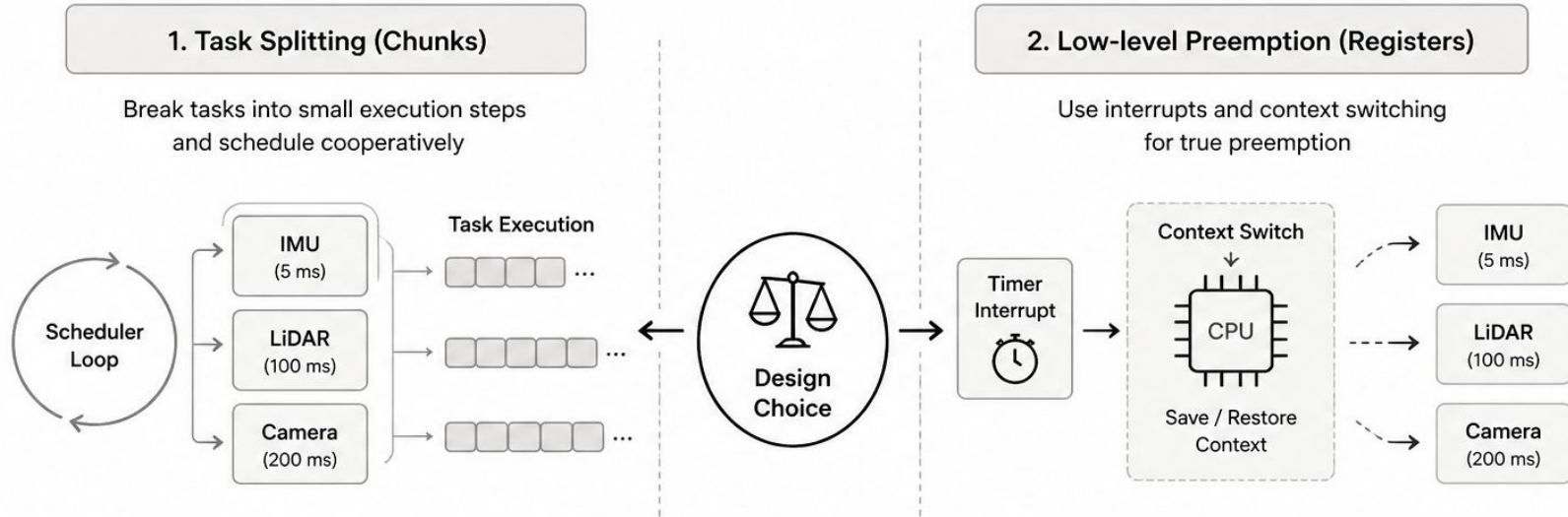
Design Choice: Implementing Preemption on STM32

2. Low-level Preemption (Registers)

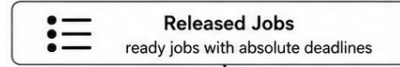
Use interrupts and context switching
for true preemption



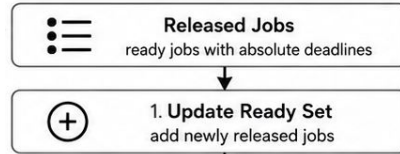
Design Choice: Implementing Preemption on STM32



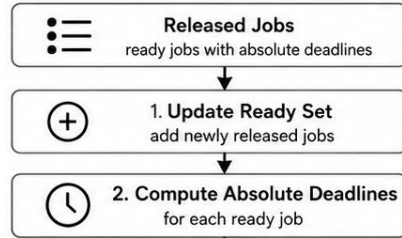
STM32 Execution Logic (EDF)



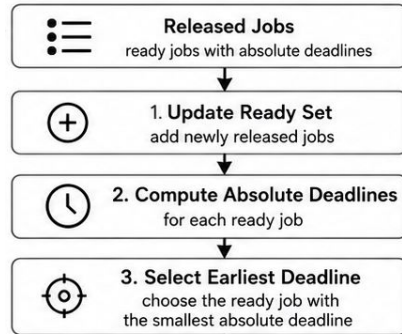
STM32 Execution Logic (EDF)



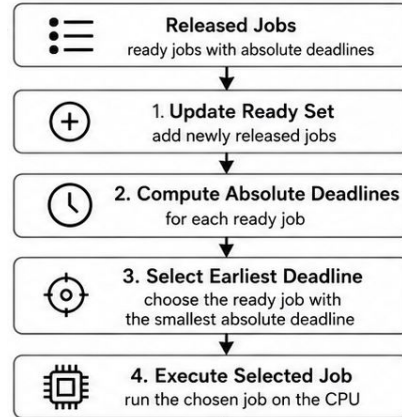
STM32 Execution Logic (EDF)



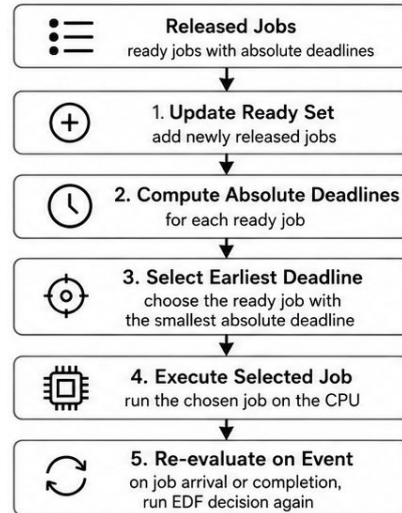
STM32 Execution Logic (EDF)



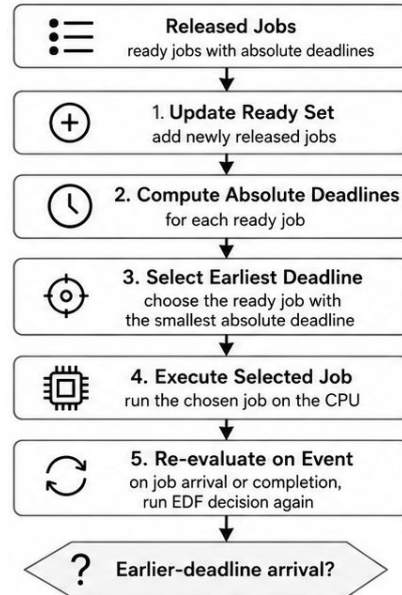
STM32 Execution Logic (EDF)



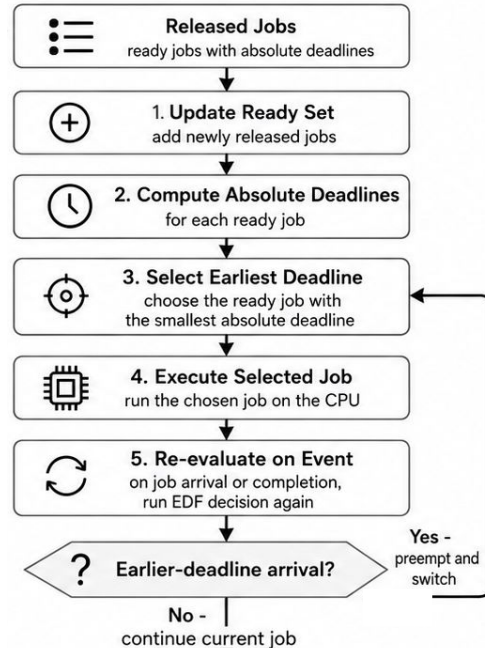
STM32 Execution Logic (EDF)



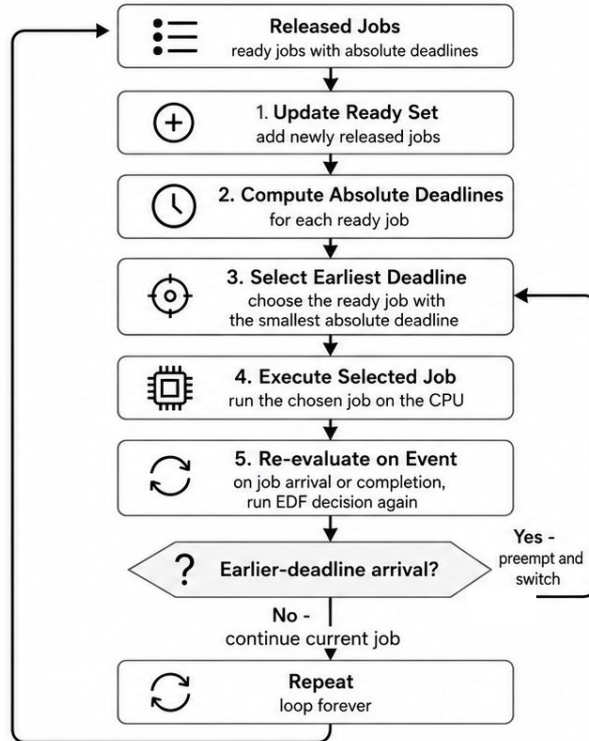
STM32 Execution Logic (EDF)



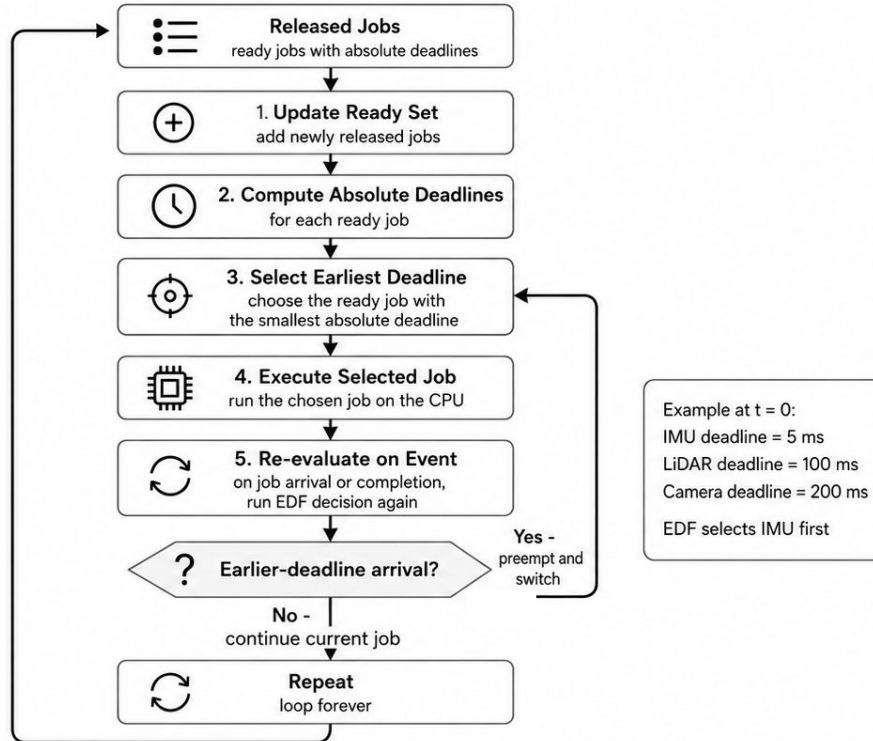
STM32 Execution Logic (EDF)



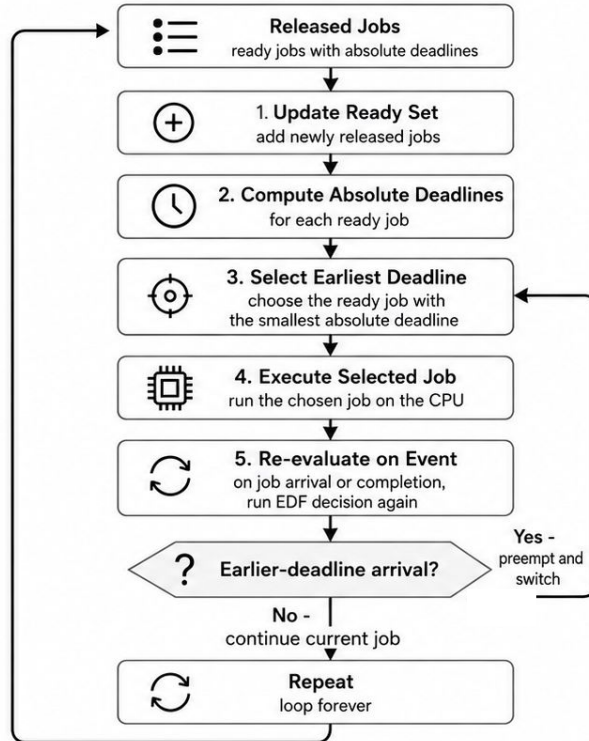
STM32 Execution Logic (EDF)



STM32 Execution Logic (EDF)



STM32 Execution Logic (EDF)



Example at $t = 0$:
IMU deadline = 5 ms
LiDAR deadline = 100 ms
Camera deadline = 200 ms
EDF selects IMU first

Key Properties

- **Dynamic priority:**
priority changes with absolute deadlines
- **EDF rule:**
earliest absolute deadline runs first
- **Preemptive model:**
a newly arrived earlier-deadline job can interrupt the current one
- **One task at a time:**
only one job uses the CPU at a given moment
- **Deadline-based scheduling:**
order is decided by deadlines, not fixed code order

Task Output

Sensor	What we measure	Processed data	Used for
--------	-----------------	----------------	----------

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance
IMU	Orientation / motion	Angle, rotation	Stabilization

Task Output

Sensor	What we measure	Processed data	Used for
LiDAR	Distance to objects	Map of surroundings	Obstacle avoidance
IMU	Orientation / motion	Angle, rotation	Stabilization
Camera	Images	Detected objects	Navigation / tracking

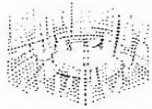
Decision Making

1. Sensor Inputs



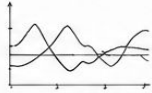
LiDAR

Measures distance
to surroundings



IMU

Measures motion
and orientation

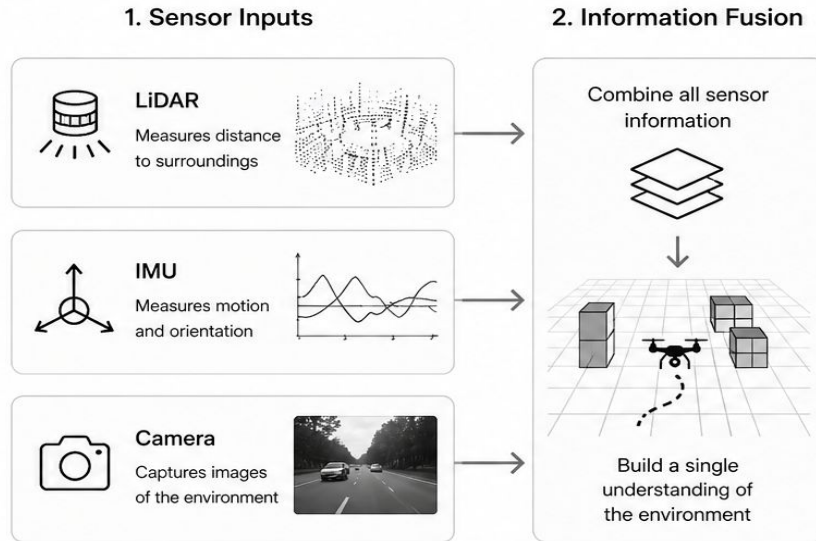


Camera

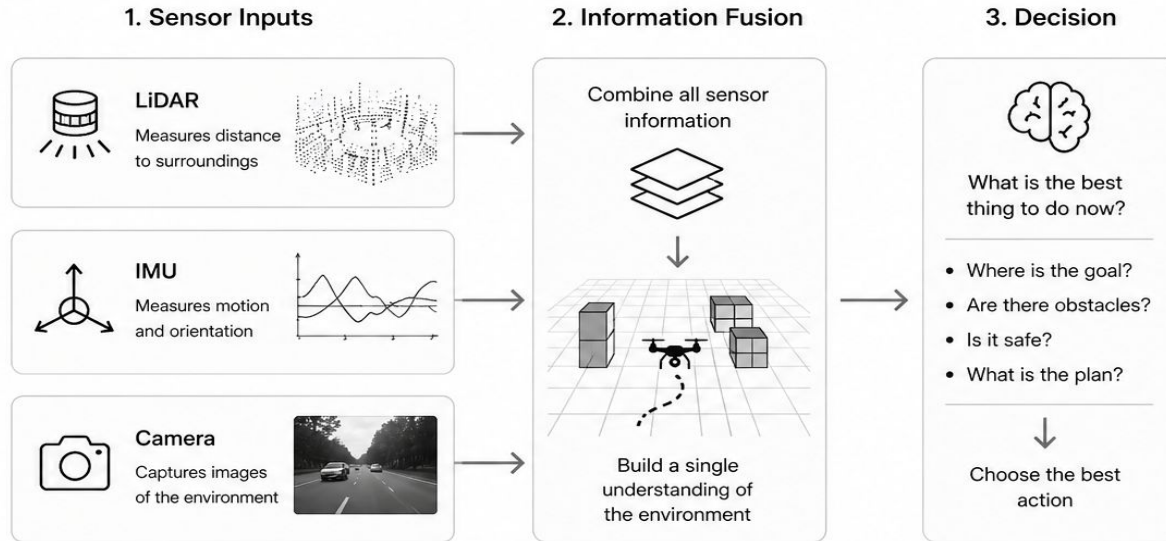
Captures images
of the environment



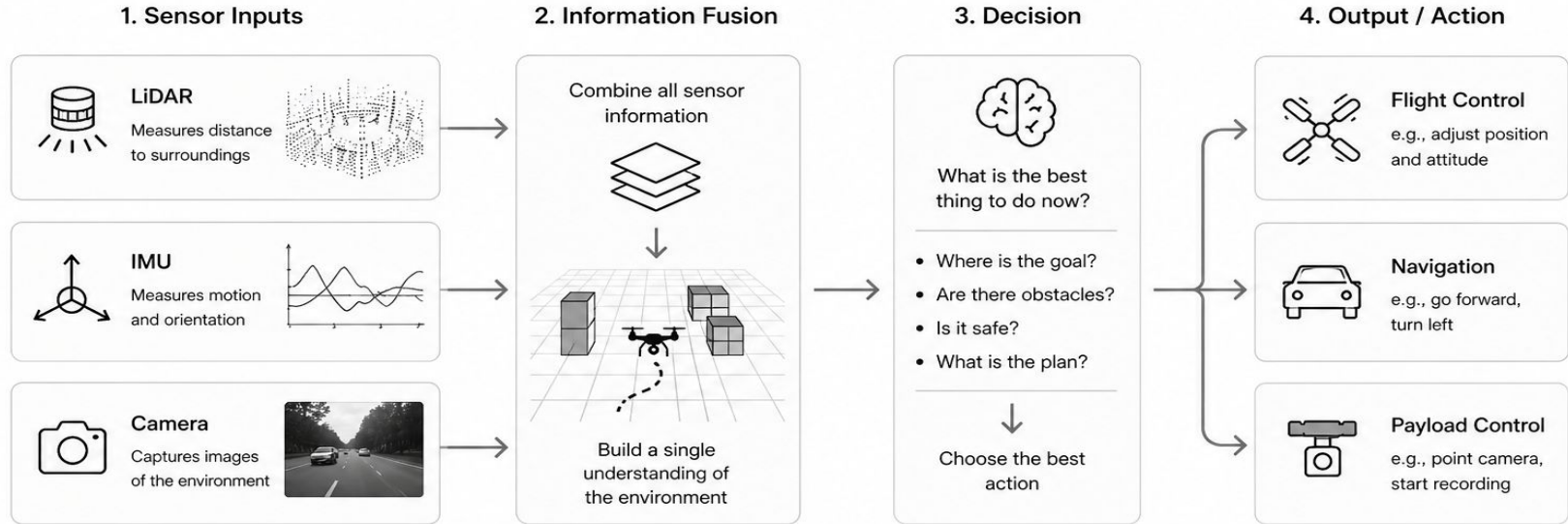
Decision Making



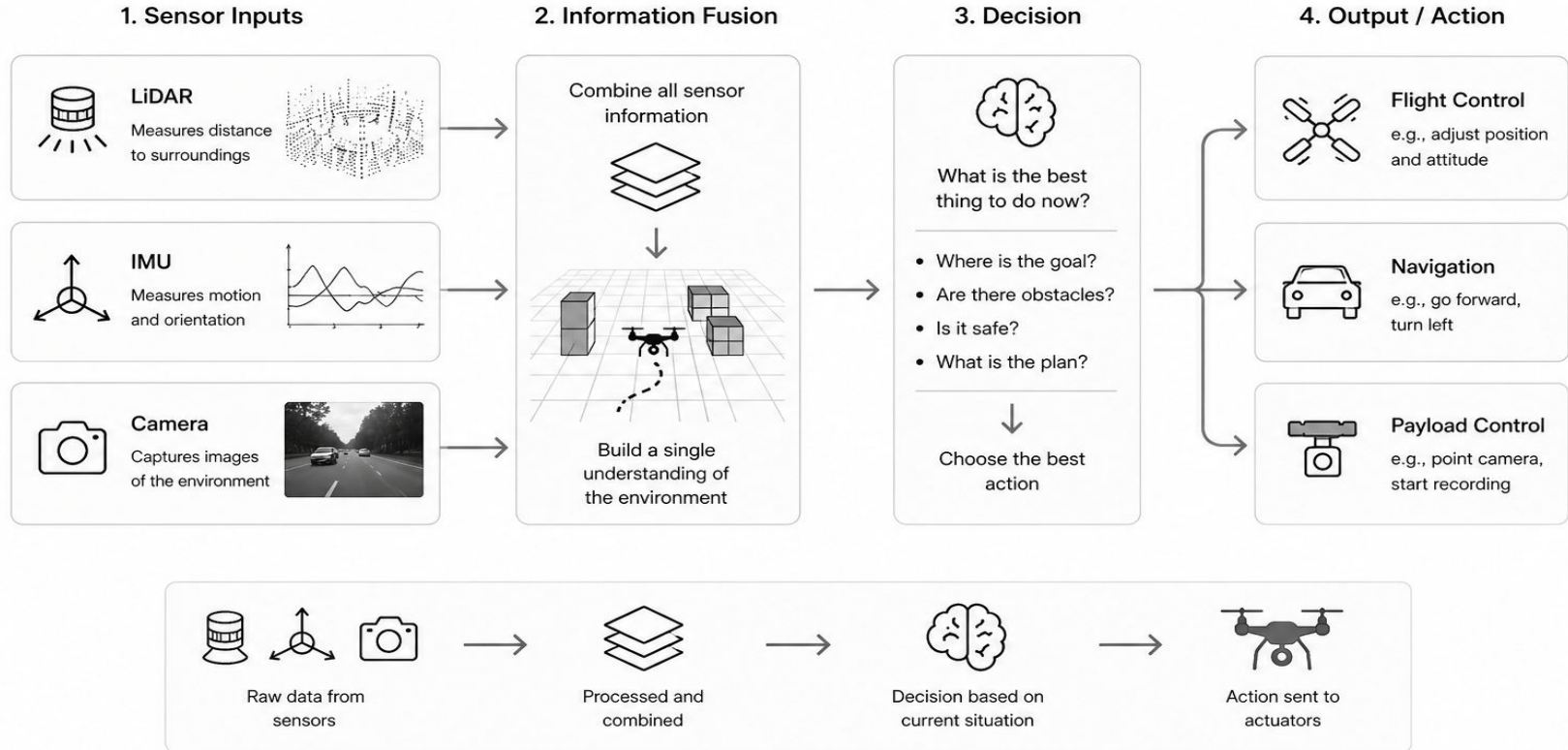
Decision Making



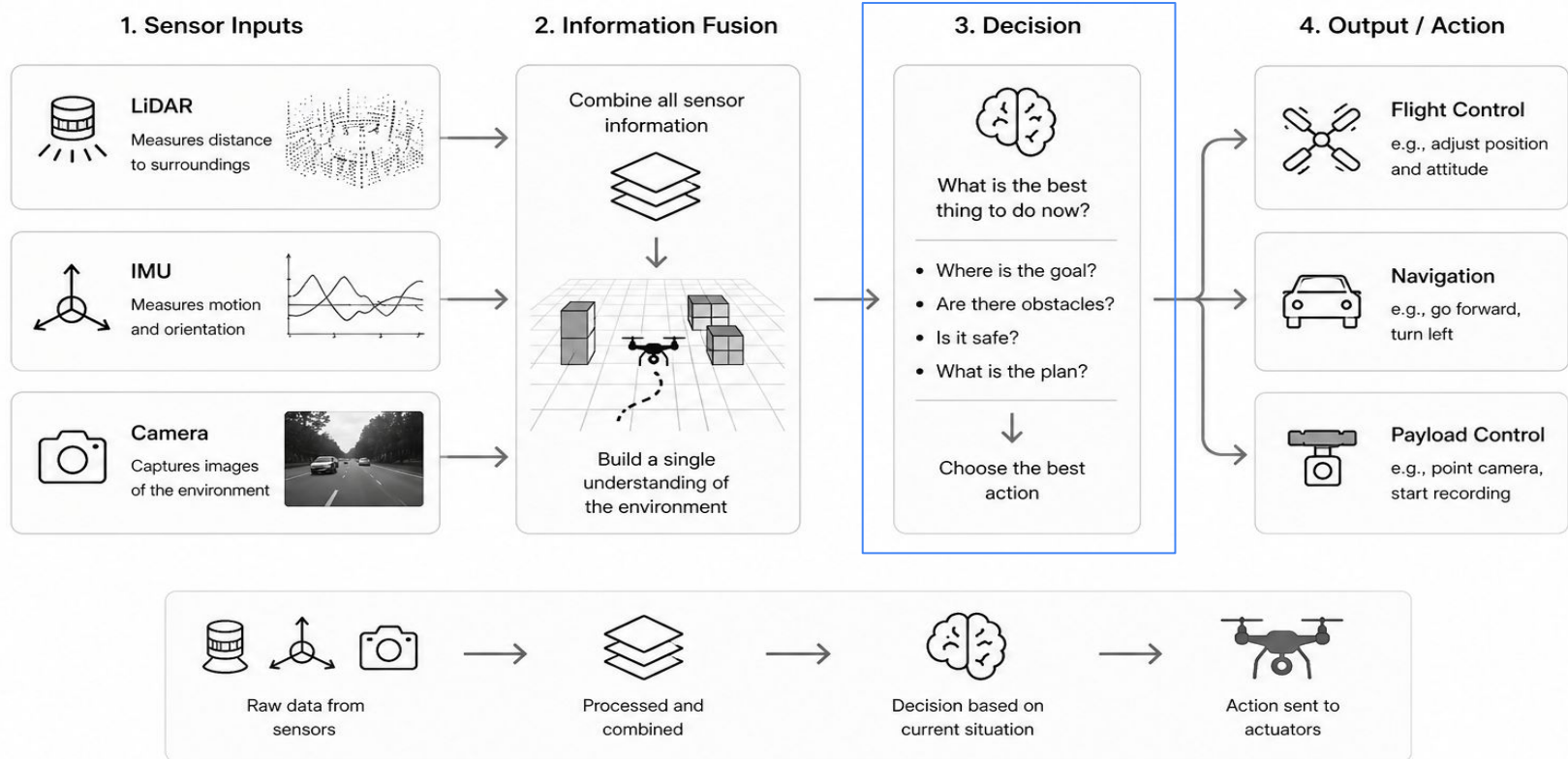
Decision Making



Decision Making



Decision Making



Control Commands

1. Decision



Based on the current
understanding of the environment.

Examples:



Obstacle ahead
→ Avoid it



Strong wind / Disturbance
→ Stabilize



Target detected
→ Track it

Control Commands

1. Decision



Based on the current understanding of the environment.

Examples:



Obstacle ahead
→ Avoid it



Strong wind / Disturbance
→ Stabilize



Target detected
→ Track it



2. Control Commands

The decision is converted into specific control commands.



Turn left
→ Change direction



Increase thrust
→ Gain speed / altitude

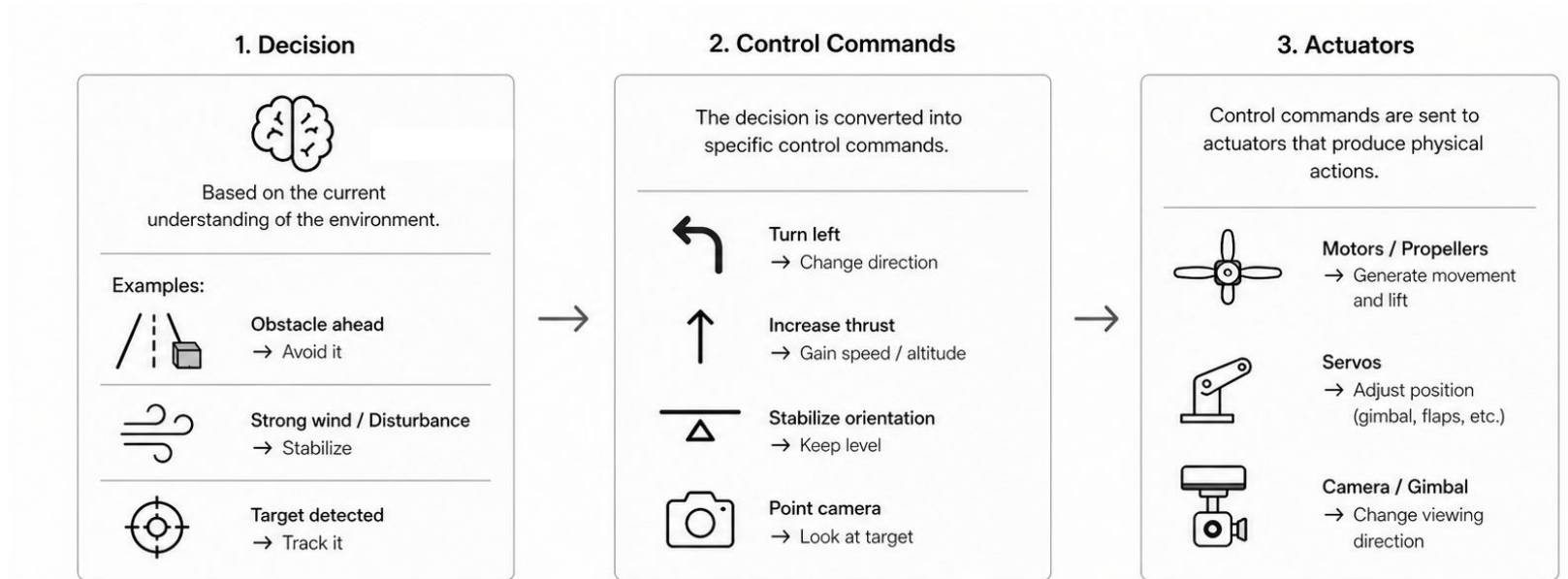


Stabilize orientation
→ Keep level



Point camera
→ Look at target

Actuators



Closed Loop System

